

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova

Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **Bidirectional Encoder Representations from Transformers**. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

1 Introduction

Language model pre-training has been shown to be effective for improving many natural language processing tasks (Dai and Le, 2015; Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018). These include sentence-level tasks such as natural language inference (Bowman et al., 2015; Williams et al., 2018) and paraphrasing (Dolan and Brockett, 2005), which aim to predict the relationships between sentences by analyzing them holistically, as well as token-level tasks such as named entity recognition and question answering, where models are required to produce fine-grained output at the token level (Tjong Kim Sang and De Meulder, 2003; Rajpurkar et al., 2016).

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers of the Transformer (Vaswani et al., 2017). Such restrictions are sub-optimal for sentence-level tasks, and could be very harmful when applying fine-tuning based approaches to token-level tasks such as question answering, where it is crucial to incorporate context from both directions.

In this paper, we improve the fine-tuning based approaches by proposing BERT: **Bidirectional Encoder Representations from Transformers**. BERT alleviates the previously mentioned unidirectionality constraint by using a “masked language model” (MLM) pre-training objective, inspired by the Cloze task (Taylor, 1953). The masked language model randomly masks some of the tokens from the input, and the objective is to predict the original vocabulary id of the masked

BERT: 预训练深度双向 Transformer 以实现语言理解

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova

Google AI 语言

{j1}jacobdevlin, mingweichang, kentonl, kristout{j2}@google.com

摘要

我们介绍了一种新的语言表示模型，称为 BERT，代表来自转换器的双向编码器表示。与最近的语言表示模型（Peters 等人，2018a；Radford 等人，2018）不同，BERT 旨在通过在所有层中联合调节左右上下文来从无标签文本中预训练深度双向表示。因此，预训练的 BERT 模型只需一个额外的输出层即可微调，以创建用于广泛任务的最新模型，例如问答和语言推理，而无需大量特定任务的架构修改。BERT 在概念上简单且在经验上强大。它在 11 项自然语言处理任务上获得了新的最新结果，包括将 GLUE 分数提高到 80.5%（绝对提升 7.7%），MultiNLI 准确率提高到 86.7%（绝对提升 4.6%），SQuAD v1.1 问答测试 F1 提高到 93.2（绝对提升 1.5 个百分点）和 SQuAD v2.0 测试 F1 提高到 83.1（绝对提升 5.1 个百分点）。

1 引言

语言模型预训练已被证明对改进许多自然语言处理任务（Dai 和 Le，2015；Peters 等人，2018a；Radford 等人，2018；Howard 和 Ruder，2018）是有效的。这些任务包括句子级别的任务，如自然语言推理（Bowman 等人，2015；Williams 等人，2018）和释义（Dolan 和 Brockett，2005），这些任务旨在通过整体分析句子来预测句子之间的关系，还包括标记级别的任务，如命名实体识别和问答，在这些任务中，模型需要在标记级别生成细粒度的输出（Tjong Kim Sang 和 De Meulder，2003；Rajpurkar 等人，2016）。

将预训练的语言表示应用于下游任务有两种现有策略：基于特征和微调。基于特征的策略，如 ELMo（Peters 等人，2018a），使用特定于任务的架构，将预训练的表示作为附加特征。微调策略，如生成式预训练转换器（OpenAI GPT）（Radford 等人，2018），引入最少的特定于任务的参数，并通过简单地微调所有预训练参数在下游任务上进行训练。这两种策略在预训练期间共享相同的目标函数，其中它们使用单向语言模型来学习通用的语言表示。

我们认为当前技术限制了预训练表示的能力，尤其是对于微调方法。主要限制是标准语言模型是单向的，这限制了在预训练期间可以使用的架构选择。例如，在 OpenAI GPT 中，作者使用了一种从左到右的架构，其中每个标记只能在前 Transformer 的自注意力层中关注之前的标记（Vaswani 等人，2017 年）。这样的限制对于句子级任务来说是不理想的，并且在将基于微调的方法应用于标记级任务（如问答）时可能会造成很大的危害，在这些任务中，结合来自两个方向的上下文至关重要。

在本文中，我们通过提出 BERT：基于转换器的双向编码器表示（Bidirectional Encoder Representations from Transformers）来改进基于微调的方法。BERT 通过使用“掩码语言模型”（MLM）预训练目标来缓解之前提到的单向性约束，该目标受 Cloze 任务（Taylor，1953）的启发。掩码语言模型会随机遮盖输入中的一些词元，目标是仅根据其上下文预测被遮盖词元的原始词汇 ID。与从左到右的语言模型预训练不同，MLM 目标使表示能够融合左右上下文，这使我们能够预训练一个深度双向 Transformer。除了掩码语言模型，我们还使用了一个“下一句预测”任务，该任务联合预训练文本对表示。本文的贡献如下：

word based only on its context. Unlike left-to-right language model pre-training, the MLM objective enables the representation to fuse the left and the right context, which allows us to pre-train a deep bidirectional Transformer. In addition to the masked language model, we also use a “next sentence prediction” task that jointly pre-trains text-pair representations. The contributions of our paper are as follows:

- We demonstrate the importance of bidirectional pre-training for language representations. Unlike Radford et al. (2018), which uses unidirectional language models for pre-training, BERT uses masked language models to enable pre-trained deep bidirectional representations. This is also in contrast to Peters et al. (2018a), which uses a shallow concatenation of independently trained left-to-right and right-to-left LMs.
- We show that pre-trained representations reduce the need for many heavily-engineered task-specific architectures. BERT is the first fine-tuning based representation model that achieves state-of-the-art performance on a large suite of sentence-level and token-level tasks, outperforming many task-specific architectures.
- BERT advances the state of the art for eleven NLP tasks. The code and pre-trained models are available at <https://github.com/google-research/bert>.

2 Related Work

There is a long history of pre-training general language representations, and we briefly review the most widely-used approaches in this section.

2.1 Unsupervised Feature-based Approaches

Learning widely applicable representations of words has been an active area of research for decades, including non-neural (Brown et al., 1992; Ando and Zhang, 2005; Blitzer et al., 2006) and neural (Mikolov et al., 2013; Pennington et al., 2014) methods. Pre-trained word embeddings are an integral part of modern NLP systems, offering significant improvements over embeddings learned from scratch (Turian et al., 2010). To pre-train word embedding vectors, left-to-right language modeling objectives have been used (Mnih and Hinton, 2009), as well as objectives to discriminate correct from incorrect words in left and right context (Mikolov et al., 2013).

These approaches have been generalized to coarser granularities, such as sentence embeddings (Kiros et al., 2015; Logeswaran and Lee, 2018) or paragraph embeddings (Le and Mikolov, 2014). To train sentence representations, prior work has used objectives to rank candidate next sentences (Jernite et al., 2017; Logeswaran and Lee, 2018), left-to-right generation of next sentence words given a representation of the previous sentence (Kiros et al., 2015), or denoising auto-encoder derived objectives (Hill et al., 2016).

ELMo and its predecessor (Peters et al., 2017, 2018a) generalize traditional word embedding research along a different dimension. They extract *context-sensitive* features from a left-to-right and a right-to-left language model. The contextual representation of each token is the concatenation of the left-to-right and right-to-left representations. When integrating contextual word embeddings with existing task-specific architectures, ELMo advances the state of the art for several major NLP benchmarks (Peters et al., 2018a) including question answering (Rajpurkar et al., 2016), sentiment analysis (Socher et al., 2013), and named entity recognition (Tjong Kim Sang and De Meulder, 2003). Melamud et al. (2016) proposed learning contextual representations through a task to predict a single word from both left and right context using LSTMs. Similar to ELMo, their model is feature-based and not deeply bidirectional. Fedus et al. (2018) shows that the cloze task can be used to improve the robustness of text generation models.

2.2 Unsupervised Fine-tuning Approaches

As with the feature-based approaches, the first works in this direction only pre-trained word embedding parameters from unlabeled text (Collobert and Weston, 2008).

More recently, sentence or document encoders which produce contextual token representations have been pre-trained from unlabeled text and fine-tuned for a supervised downstream task (Dai and Le, 2015; Howard and Ruder, 2018; Radford et al., 2018). The advantage of these approaches is that few parameters need to be learned from scratch. At least partly due to this advantage, OpenAI GPT (Radford et al., 2018) achieved previously state-of-the-art results on many sentence-level tasks from the GLUE benchmark (Wang et al., 2018a). Left-to-right language model-

与左到右的语言模型预训练不同, MLM目标使表示能够融合左右上下文, 这使我们能够预训练一个深度双向Transformer。除了掩码语言模型, 我们还使用了一个“下一句预测”任务, 该任务联合预训练文本对表示。本文的贡献如下:

- 我们展示了双向预训练对语言表示的重要性。与Radford等人(2018年)不同, 后者使用单向语言模型进行预训练, BERT使用掩码语言模型以实现预训练的深度双向表示。这也与Peters等人(2018a)形成对比, 后者使用独立训练的从左到右和从右到左的LM的浅层连接。
- 我们表明预训练表示减少了对于许多高度工程化的特定任务架构的需求。BERT是第一个基于微调的表示模型, 在大型套件上实现了最先进的性能。
of sentence-level and token-level tasks, outperforming many task-specific architectures.
- BERT在十一个NLP任务上推进了最先进的技术。代码和预训练模型可在 <https://github.com/google-research/bert> 获取。

2 相关工作

存在预训练通用语言表示的悠久历史, 在本节中, 我们简要回顾了最广泛使用的方法。

2.1 无监督特征方法

学习词的广泛适用表示已经是一个活跃的研究领域几十年来, 包括非神经 (Brown et al., 1992; Ando and Zhang, 2005; Blitzer et al., 2006) 和神经 (Mikolov et al., 2013; Pennington et al., 2014) 方法。预训练词嵌入是现代NLP系统的重要组成部分, 比从头开始学习的嵌入提供了显著的改进 (Turian et al., 2010)。为了预训练词嵌入向量, 已经使用过从左到右的语言建模目标 (Mnih and Hinton, 2009), 以及区分左右上下文中正确和错误单词的目标 (Mikolov et al., 2013)。

这些方法已经被推广到更粗的粒度, 例如句子嵌入 (Kiros et al., 2015; Logeswaran and Lee, 2018) 或段落嵌入 (Le and Mikolov, 2014)。为了训练句子表示, 先前工作使用了排名候选下一句的目标 (Jernite et al., 2017; Logeswaran and Lee, 2018), 给定先前句子的表示时从左到右生成下一句单词 (Kiros et al., 2015), 或基于降噪自编码器衍生的目标 (Hill et al., 2016)。

ELMo及其前身 (Peters等人, 2017年, 2018a年) 在传统词嵌入研究的不同维度上进行了泛化。它们从左到右和从右到左的语言模型中提取上下文相关的特征。每个标记的上下文表示是左到右和右到左表示的连接。当将上下文词嵌入与现有的特定任务架构集成时, ELMo在几个主要的NLP基准测试 (Peters等人, 2018a年) 上推进了技术状态, 包括问答 (Rajpurkar等人, 2016年)、情感分析 (Socher等人, 2013年) 和命名实体识别 (Tjong Kim Sang和De Meulder, 2003年)。Melamud等人 (2016年) 提出了通过任务学习上下文表示, 以使用LSTM从左右上下文中预测单个词。与ELMo类似, 他们的模型是基于特征的, 而不是深度双向的。Fedus等人 (2018年) 表明, 完形填空任务可以用来提高文本生成模型的鲁棒性。

2.2 无监督微调方法

与基于特征的方法类似, 这个方向上的第一个工作仅从无标签文本中预训练词嵌入参数 (Collobert and Weston, 2008)。

最近, 用于生成上下文标记表示的句子或文档编码器已被从无标签文本中预训练, 并为监督下游任务进行微调 (Dai和Le, 2015; Howard和Ruder, 2018; Radford等人, 2018)。这些方法的优点是需从头开始学习很少的参数。至少部分由于这个优点, OpenAI GPT (Radford等人, 2018) 在GLUE基准测试的许多句子级任务上之前达到了最先进的结果 (Wang等人, 2018a)。从左到右的语言模型-

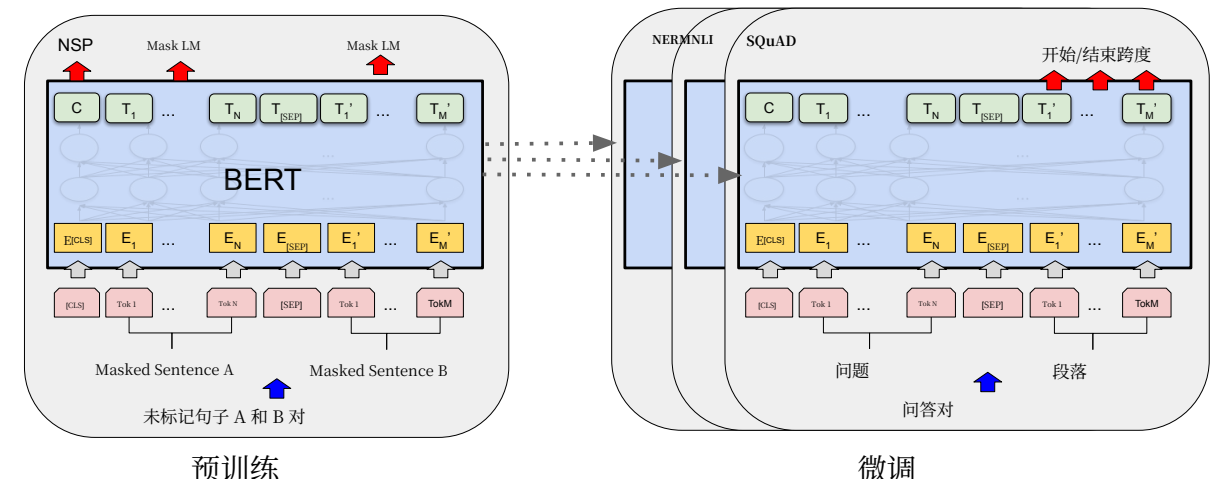
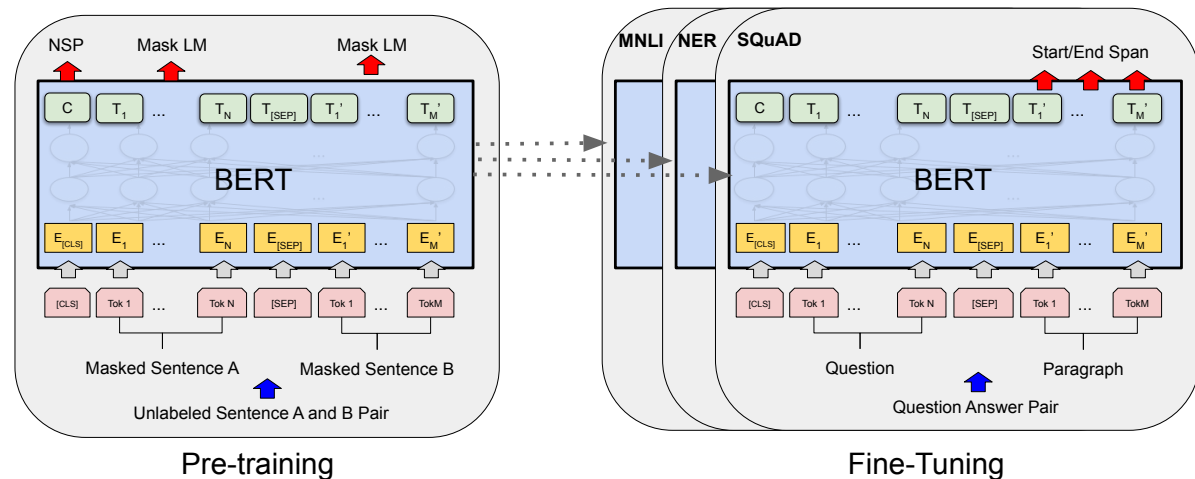


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

ing and auto-encoder objectives have been used for pre-training such models (Howard and Ruder, 2018; Radford et al., 2018; Dai and Le, 2015).

2.3 Transfer Learning from Supervised Data

There has also been work showing effective transfer from supervised tasks with large datasets, such as natural language inference (Conneau et al., 2017) and machine translation (McCann et al., 2017). Computer vision research has also demonstrated the importance of transfer learning from large pre-trained models, where an effective recipe is to fine-tune models pre-trained with ImageNet (Deng et al., 2009; Yosinski et al., 2014).

3 BERT

We introduce BERT and its detailed implementation in this section. There are two steps in our framework: *pre-training* and *fine-tuning*. During pre-training, the model is trained on unlabeled data over different pre-training tasks. For fine-tuning, the BERT model is first initialized with the pre-trained parameters, and all of the parameters are fine-tuned using labeled data from the downstream tasks. Each downstream task has separate fine-tuned models, even though they are initialized with the same pre-trained parameters. The question-answering example in Figure 1 will serve as a running example for this section.

A distinctive feature of BERT is its unified architecture across different tasks. There is mini-

mal difference between the pre-trained architecture and the final downstream architecture.

Model Architecture BERT’s model architecture is a multi-layer bidirectional Transformer encoder based on the original implementation described in Vaswani et al. (2017) and released in the `tensor2tensor` library.¹ Because the use of Transformers has become common and our implementation is almost identical to the original, we will omit an exhaustive background description of the model architecture and refer readers to Vaswani et al. (2017) as well as excellent guides such as “The Annotated Transformer.”²

In this work, we denote the number of layers (i.e., Transformer blocks) as L , the hidden size as H , and the number of self-attention heads as A .³ We primarily report results on two model sizes: **BERT_{BASE}** ($L=12$, $H=768$, $A=12$, Total Parameters=110M) and **BERT_{LARGE}** ($L=24$, $H=1024$, $A=16$, Total Parameters=340M).

BERT_{BASE} was chosen to have the same model size as OpenAI GPT for comparison purposes. Critically, however, the BERT Transformer uses bidirectional self-attention, while the GPT Transformer uses constrained self-attention where every token can only attend to context to its left.⁴

¹<https://github.com/tensorflow/tensor2tensor>

²<http://nlp.seas.harvard.edu/2018/04/03/attention.html>

³In all cases we set the feed-forward/filter size to be $4H$, i.e., 3072 for the $H = 768$ and 4096 for the $H = 1024$.

⁴We note that in the literature the bidirectional Trans-

和自动编码器目标已被用于预训练此类模型（Howard和Ruder, 2018; Radford et al., 2018; Dai and Le, 2015）。

2.3 从监督数据中进行迁移学习

也有研究表明，从具有大型数据集的监督任务中可以有效地进行迁移学习，例如自然语言推理（Conneau等人，2017年）和机器翻译（McCann等人，2017年）。计算机视觉研究也证明了从大型预训练模型进行迁移学习的重要性，其中有效的方法是使用ImageNet预训练的模型进行微调（Deng等人，2009年；Yosinski等人，2014年）。

3 BERT

我们在本节中介绍了BERT及其详细实现。我们的框架中有两个步骤：预训练和微调。在预训练期间，模型在未标记数据上进行不同预训练任务的训练。对于微调，BERT模型首先使用预训练参数初始化，然后使用下游任务的标记数据进行所有参数的微调。尽管它们使用相同的预训练参数初始化，但每个下游任务都有单独的微调模型。图1中的问答示例将作为本节的运行示例。

BERT的一个显著特点是其在不同任务中采用的统一架构。预训练架构和最终下游架构之间的差异很小。

预训练架构和最终下游架构之间的差异很小。

模型架构BERT的模型架构是基于Vaswani等人（2017年）中描述的原始实现并在 `tensor2tensor` 库中发布的多层双向Transformer编码器。¹ 由于Transformer的使用已变得普遍且我们的实现几乎与原始实现相同，我们将省略模型架构的详尽背景描述，并建议读者参考Vaswani等人（2017年）以及“Annotated Transformer”等优秀指南。²

在这项工作中，我们将层数（即Transformer模块）记为 L ，隐藏层大小记为 H ，自注意力头的数量记为 A 。³ 我们主要报告了两种模型大小的结果：BERT_{BASE} ($L=12$, $H=768$, $A=12$, 总参数=110M) 和 BERT_{LARGE} ($L=24$, $H=1024$, $A=16$, 总参数=340M)。

BERT_{BASE} 被选用来与OpenAI GPT具有相同的模型大小，以便进行比较。然而，关键在于，BERT Transformer使用双向自注意力机制，而GPT Transformer使用受限自注意力机制，其中每个token只能关注其左侧的上下文。⁴

¹<https://github.com/tensorflow/tensor2tensor>

²<http://nlp.seas.harvard.edu/2018/04/03/attention.html>

³In all cases we set the feed-forward/filter size to be $4H$, i.e., 3072 for $H = 768$ 和 4096 对于 $H = 1024$ 。我们注意到在文献中双向Transformer-

Input/Output Representations To make BERT handle a variety of down-stream tasks, our input representation is able to unambiguously represent both a single sentence and a pair of sentences (e.g., $\langle \text{Question, Answer} \rangle$) in one token sequence. Throughout this work, a “sentence” can be an arbitrary span of contiguous text, rather than an actual linguistic sentence. A “sequence” refers to the input token sequence to BERT, which may be a single sentence or two sentences packed together.

We use WordPiece embeddings (Wu et al., 2016) with a 30,000 token vocabulary. The first token of every sequence is always a special classification token ($[\text{CLS}]$). The final hidden state corresponding to this token is used as the aggregate sequence representation for classification tasks. Sentence pairs are packed together into a single sequence. We differentiate the sentences in two ways. First, we separate them with a special token ($[\text{SEP}]$). Second, we add a learned embedding to every token indicating whether it belongs to sentence A or sentence B. As shown in Figure 1, we denote input embedding as E , the final hidden vector of the special $[\text{CLS}]$ token as $C \in \mathbb{R}^H$, and the final hidden vector for the i^{th} input token as $T_i \in \mathbb{R}^H$.

For a given token, its input representation is constructed by summing the corresponding token, segment, and position embeddings. A visualization of this construction can be seen in Figure 2.

3.1 Pre-training BERT

Unlike Peters et al. (2018a) and Radford et al. (2018), we do not use traditional left-to-right or right-to-left language models to pre-train BERT. Instead, we pre-train BERT using two unsupervised tasks, described in this section. This step is presented in the left part of Figure 1.

Task #1: Masked LM Intuitively, it is reasonable to believe that a deep bidirectional model is strictly more powerful than either a left-to-right model or the shallow concatenation of a left-to-right and a right-to-left model. Unfortunately, standard conditional language models can only be trained left-to-right or right-to-left, since bidirectional conditioning would allow each word to indirectly “see itself”, and the model could trivially predict the target word in a multi-layered context.

former is often referred to as a “Transformer encoder” while the left-context-only version is referred to as a “Transformer decoder” since it can be used for text generation.

In order to train a deep bidirectional representation, we simply mask some percentage of the input tokens at random, and then predict those masked tokens. We refer to this procedure as a “masked LM” (MLM), although it is often referred to as a *Cloze* task in the literature (Taylor, 1953). In this case, the final hidden vectors corresponding to the mask tokens are fed into an output softmax over the vocabulary, as in a standard LM. In all of our experiments, we mask 15% of all WordPiece tokens in each sequence at random. In contrast to denoising auto-encoders (Vincent et al., 2008), we only predict the masked words rather than reconstructing the entire input.

Although this allows us to obtain a bidirectional pre-trained model, a downside is that we are creating a mismatch between pre-training and fine-tuning, since the $[\text{MASK}]$ token does not appear during fine-tuning. To mitigate this, we do not always replace “masked” words with the actual $[\text{MASK}]$ token. The training data generator chooses 15% of the token positions at random for prediction. If the i -th token is chosen, we replace the i -th token with (1) the $[\text{MASK}]$ token 80% of the time (2) a random token 10% of the time (3) the unchanged i -th token 10% of the time. Then, T_i will be used to predict the original token with cross entropy loss. We compare variations of this procedure in Appendix C.2.

Task #2: Next Sentence Prediction (NSP)

Many important downstream tasks such as Question Answering (QA) and Natural Language Inference (NLI) are based on understanding the *relationship* between two sentences, which is not directly captured by language modeling. In order to train a model that understands sentence relationships, we pre-train for a binarized *next sentence prediction* task that can be trivially generated from any monolingual corpus. Specifically, when choosing the sentences A and B for each pre-training example, 50% of the time B is the actual next sentence that follows A (labeled as IsNext), and 50% of the time it is a random sentence from the corpus (labeled as NotNext). As we show in Figure 1, C is used for next sentence prediction (NSP).⁵ Despite its simplicity, we demonstrate in Section 5.1 that pre-training towards this task is very beneficial to both QA and NLI.⁶

⁵The final model achieves 97%-98% accuracy on NSP.

⁶The vector C is not a meaningful sentence representation without fine-tuning, since it was trained with NSP.

输入/输出表示 为了使BERT能够处理各种下游任务, 我们的输入表示能够在一个token序列中明确地表示单个句子和句子对 (例如, $\langle \text{问题, 答案} \rangle$)。在整个工作中, “句子”可以是指任意连续文本的任意跨度, 而不仅仅是实际的句子。“序列”是指输入到BERT的token序列, 它可能是一个句子或两个句子打包在一起。

我们使用 WordPiece 嵌入 (Wu 等人, 2016 年) 和 30,000 个词的词汇表。每个序列的第一个标记始终是一个特殊的分类标记 ($[\text{CLS}]$)。与此标记对应的最终隐藏状态用作分类任务的聚合序列表示。句子对被打包成一个序列。我们以两种方式区分句子。首先, 我们用特殊标记 ($[\text{SEP}]$) 将它们分开。其次, 我们为每个标记添加一个学习的嵌入, 以指示它属于句子 A 还是句子 B。如图 1 所示, 我们将输入嵌入表示为 E , 特殊 $[\text{CLS}]$ 标记的最终隐藏向量表示为 $C \in \mathbb{R}^H$, i^{th} 输入标记的最终隐藏向量表示为 $T_i \in \mathbb{R}^H$ 。

对于给定的标记, 其输入表示是通过将相应的标记、片段和位置嵌入相加构建的。这种构建的可视化可以在图 2 中看到。

3.1 预训练BERT

与Peters等人(2018a)和Radford等人(2018)不同, 我们不使用传统的从左到右或从右到左的语言模型来预训练BERT。相反, 我们使用本节中描述的两个无监督任务来预训练 BERT。这一步骤在图1的左侧部分展示。

任务 #1: 掩码语言模型 直觉上, 可以合理地认为深度双向模型比左到右模型或浅层左到右和右到左模型的组合更强大。不幸的是, 标准条件语言模型只能从左到右或从右到左进行训练, 因为双向条件允许每个词间接 “看到自己”, 并且模型可以在多层上下文中轻易预测目标词。

前者通常被称为 “Transformer编码器”, 而仅使用左上上下文的版本被称为 “Transformer解码器”, 因为它可用于文本生成。

为了训练深度双向表示, 我们简单地随机掩码输入标记的一定百分比, 然后预测这些掩码标记。我们将此过程称为 “掩码语言模型” (MLM), 尽管在文献中通常称为完形填空任务 (Taylor, 1953)。在这种情况下, 与掩码标记对应的最终隐藏向量被输入到词汇表上的输出softmax中, 就像标准语言模型一样。在我们的所有实验中, 我们随机掩码每个序列中所有WordPiece标记的15%。与去噪自编码器 (Vincent 等人, 2008) 不同, 我们只预测掩码词而不是重建整个输入。

虽然这使我们能够获得一个双向预训练模型, 但缺点是我们造成了预训练和微调之间的不匹配, 因为 $[\text{MASK}]$ 标记在微调期间不会出现。为了缓解这个问题, 我们不总是用实际的 $[\text{MASK}]$ 标记替换 “被掩盖” 的单词。训练数据生成器随机选择15%的标记位置进行预测。如果选择了第 i 个标记, 我们将第 i 个标记替换为: (1) 80%的时间用 $[\text{MASK}]$ 标记 (2) 10%的时间用一个随机标记 (3) 10%的时间保持第 i 个标记不变。然后, 使用 T_i 通过交叉熵损失预测原始标记。我们在附录C.2中比较了此过程的变体。

任务 #2: 下一句预测 (NSP) 许多重要的下游任务, 如问答 (QA) 和自然语言推理 (NLI), 都基于理解两个句子之间的关系, 而语言模型并不能直接捕捉这种关系。为了训练一个理解句子关系的模型, 我们预训练了一个二分类的下一句预测任务, 该任务可以轻易地从任何单语语料库中生成。具体来说, 在选择每个预训练样本的句子A和B时, 50% 的时间B是紧随A的实际下一句 (标记为 IsNext), 而另外 50% 的时间它是从语料库中随机选择的句子 (标记为 NotNext)。如图1所示, C 用于下一句预测 (NSP)。⁵ 尽管非常简单, 我们在第 5.1 节中证明了针对该任务的预训练对 QA 和 NLI 都非常有益。⁶

⁵最终模型在NSP上达到了97%-98%的准确率。⁶未经微调, 向量 C 不能作为有意义的句子表示, 因为它是在NSP上训练的。

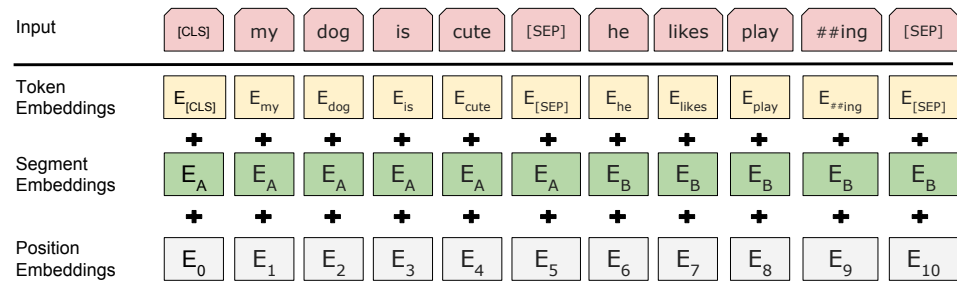


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

The NSP task is closely related to representation-learning objectives used in Jernite et al. (2017) and Logeswaran and Lee (2018). However, in prior work, only sentence embeddings are transferred to down-stream tasks, where BERT transfers all parameters to initialize end-task model parameters.

Pre-training data The pre-training procedure largely follows the existing literature on language model pre-training. For the pre-training corpus we use the BooksCorpus (800M words) (Zhu et al., 2015) and English Wikipedia (2,500M words). For Wikipedia we extract only the text passages and ignore lists, tables, and headers. It is critical to use a document-level corpus rather than a shuffled sentence-level corpus such as the Billion Word Benchmark (Chelba et al., 2013) in order to extract long contiguous sequences.

3.2 Fine-tuning BERT

Fine-tuning is straightforward since the self-attention mechanism in the Transformer allows BERT to model many downstream tasks—whether they involve single text or text pairs—by swapping out the appropriate inputs and outputs. For applications involving text pairs, a common pattern is to independently encode text pairs before applying bidirectional cross attention, such as Parikh et al. (2016); Seo et al. (2017). BERT instead uses the self-attention mechanism to unify these two stages, as encoding a concatenated text pair with self-attention effectively includes *bidirectional* cross attention between two sentences.

For each task, we simply plug in the task-specific inputs and outputs into BERT and fine-tune all the parameters end-to-end. At the input, sentence A and sentence B from pre-training are analogous to (1) sentence pairs in paraphrasing, (2) hypothesis-premise pairs in entailment, (3) question-passage pairs in question answering, and

(4) a degenerate text- $\emptyset$ pair in text classification or sequence tagging. At the output, the token representations are fed into an output layer for token-level tasks, such as sequence tagging or question answering, and the [CLS] representation is fed into an output layer for classification, such as entailment or sentiment analysis.

Compared to pre-training, fine-tuning is relatively inexpensive. All of the results in the paper can be replicated in at most 1 hour on a single Cloud TPU, or a few hours on a GPU, starting from the exact same pre-trained model.⁷ We describe the task-specific details in the corresponding subsections of Section 4. More details can be found in Appendix A.5.

4 Experiments

In this section, we present BERT fine-tuning results on 11 NLP tasks.

4.1 GLUE

The General Language Understanding Evaluation (GLUE) benchmark (Wang et al., 2018a) is a collection of diverse natural language understanding tasks. Detailed descriptions of GLUE datasets are included in Appendix B.1.

To fine-tune on GLUE, we represent the input sequence (for single sentence or sentence pairs) as described in Section 3, and use the final hidden vector $C \in \mathbb{R}^H$ corresponding to the first input token ([CLS]) as the aggregate representation. The only new parameters introduced during fine-tuning are classification layer weights $W \in \mathbb{R}^{K \times H}$, where K is the number of labels. We compute a standard classification loss with C and W , i.e., $\log(\text{softmax}(CW^T))$.

⁷For example, the BERT SQuAD model can be trained in around 30 minutes on a single Cloud TPU to achieve a Dev F1 score of 91.0%.

⁸See (10) in <https://gluebenchmark.com/faq>.

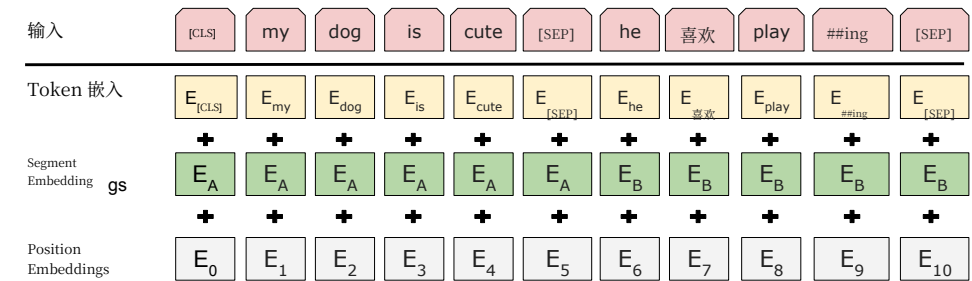


图2: BERT输入表示。输入嵌入是词嵌入、段嵌入和位置嵌入的和。

NSP 任务与 Jernite 等人 (2017) 和 Logeswaran 和 Lee (2018) 中使用的表示学习目标密切相关。然而，在之前的工作中，只有句子嵌入被迁移到下游任务，而 BERT 将所有参数迁移以初始化最终任务的模型参数。

预训练数据 预训练过程基本遵循现有的语言模型预训练文献。对于预训练语料库，我们使用了 BooksCorpus (8亿词) (Zhu 等人, 2015) 和 English Wikipedia (25亿词)。对于 Wikipedia，我们只提取文本段落，忽略列表、表格和标题。使用文档级语料库而不是像 Billion Word Benchmark (Chelba 等人, 2013) 这样的句子级打乱语料库对于提取长连续序列至关重要。

3.2 微调 BERT

微调很简单，因为 Transformer 中的自注意力机制允许 BERT 通过替换适当的输入和输出来对许多下游任务进行建模——无论是涉及单个文本还是文本对。对于涉及文本对的应用，一个常见的模式是独立编码文本对，然后再应用双向交叉注意力，例如 Parikh 等人 (2016)；Seo 等人 (2017)。BERT 相反使用自注意力机制来统一这两个阶段，因为使用自注意力机制对连接的文本对进行编码实际上在两个句子之间包含了双向交叉注意力。

对于每个任务，我们只需将特定任务的输入和输出输入到 BERT 中，并端到端微调所有参数。在输入端，预训练中的句子 A 和句子 B 类似于 (1) 释义中的句子对，(2) 蕴含中的假设-前提对，(3) 问答中的问题-段落对，以及

(4) 文本分类或序列标记中退化的文本- $\emptyset$ 对。在输出端，标记表示被输入到一个输出层用于标记级任务，例如序列标记或问答，而 [CLS] 表示被输入到一个输出层用于分类任务，例如蕴含或情感分析。

与预训练相比，微调的成本相对较低。论文中的所有结果都可以在单个 Cloud TPU 上最多 1 小时内复制，或在 GPU 上几小时内，从完全相同的预训练模型开始。⁷ 我们在第 4 章的相关小节中描述了特定任务的详细信息。更多细节可以在附录 A.5 中找到。

4 实验

在本节中，我们展示了在 11 个 NLP 任务上的 BERT 微调结果。

4.1 GLUE

通用语言理解评估 (GLUE) 基准 (Wang 等人, 2018a) 是一系列多样化的自然语言理解任务。GLUE 数据集的详细描述包括在附录 B.1 中。

要在 GLUE 上微调，我们按照第 3 节所述将输入序列（用于单句或句子对）表示为，并使用与第一个输入标记 ([CLS]) 对应的最终隐藏向量 $C \in \mathbb{R}^H$ 作为聚合表示。在微调过程中引入的唯一新参数是分类层权重 $W \in \mathbb{R}^{K \times H}$ ，其中 K 是标签的数量。我们使用 C 和 W 计算标准的分类损失，即 $\log(\text{softmax}(CW^T))$ 。

⁷例如，BERT SQuAD 模型可以在单个 Cloud TPU 上大约 30 分钟内训练，以实现 91.0% 的 Dev F1 分数。

⁸参见 <https://gluebenchmark.com/faq> 中的 (10)。

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The “Average” column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.⁸ BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

We use a batch size of 32 and fine-tune for 3 epochs over the data for all GLUE tasks. For each task, we selected the best fine-tuning learning rate (among 5e-5, 4e-5, 3e-5, and 2e-5) on the Dev set. Additionally, for BERT_{LARGE} we found that fine-tuning was sometimes unstable on small datasets, so we ran several random restarts and selected the best model on the Dev set. With random restarts, we use the same pre-trained checkpoint but perform different fine-tuning data shuffling and classifier layer initialization.⁹

Results are presented in Table 1. Both BERT_{BASE} and BERT_{LARGE} outperform all systems on all tasks by a substantial margin, obtaining 4.5% and 7.0% respective average accuracy improvement over the prior state of the art. Note that BERT_{BASE} and OpenAI GPT are nearly identical in terms of model architecture apart from the attention masking. For the largest and most widely reported GLUE task, MNLI, BERT obtains a 4.6% absolute accuracy improvement. On the official GLUE leaderboard¹⁰, BERT_{LARGE} obtains a score of 80.5, compared to OpenAI GPT, which obtains 72.8 as of the date of writing.

We find that BERT_{LARGE} significantly outperforms BERT_{BASE} across all tasks, especially those with very little training data. The effect of model size is explored more thoroughly in Section 5.2.

4.2 SQuAD v1.1

The Stanford Question Answering Dataset (SQuAD v1.1) is a collection of 100k crowd-sourced question/answer pairs (Rajpurkar et al., 2016). Given a question and a passage from

⁹The GLUE data set distribution does not include the Test labels, and we only made a single GLUE evaluation server submission for each of BERT_{BASE} and BERT_{LARGE}.

¹⁰<https://gluebenchmark.com/leaderboard>

Wikipedia containing the answer, the task is to predict the answer text span in the passage.

As shown in Figure 1, in the question answering task, we represent the input question and passage as a single packed sequence, with the question using the A embedding and the passage using the B embedding. We only introduce a start vector $S \in \mathbb{R}^H$ and an end vector $E \in \mathbb{R}^H$ during fine-tuning. The probability of word i being the start of the answer span is computed as a dot product between T_i and S followed by a softmax over all of the words in the paragraph: $P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$. The analogous formula is used for the end of the answer span. The score of a candidate span from position i to position j is defined as $S \cdot T_i + E \cdot T_j$, and the maximum scoring span where $j \geq i$ is used as a prediction. The training objective is the sum of the log-likelihoods of the correct start and end positions. We fine-tune for 3 epochs with a learning rate of 5e-5 and a batch size of 32.

Table 2 shows top leaderboard entries as well as results from top published systems (Seo et al., 2017; Clark and Gardner, 2018; Peters et al., 2018a; Hu et al., 2018). The top results from the SQuAD leaderboard do not have up-to-date public system descriptions available,¹¹ and are allowed to use any public data when training their systems. We therefore use modest data augmentation in our system by first fine-tuning on TriviaQA (Joshi et al., 2017) before fine-tuning on SQuAD.

Our best performing system outperforms the top leaderboard system by +1.5 F1 in ensembling and +1.3 F1 as a single system. In fact, our single BERT model outperforms the top ensemble system in terms of F1 score. Without TriviaQA fine-

¹¹QANet is described in Yu et al. (2018), but the system has improved substantially after publication.

系统	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	平均 -
OpenAI 之前的 SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

表1: GLUE测试结果, 由评估服务器评分 (<https://gluebenchmark.com/leaderboard>)。每个任务下方的数字表示训练样本数量。“平均”列与官方GLUE分数略有不同, 因为我们排除了有问题的WNLI集。⁸ BERT和OpenAI GPT是单模型、单任务。对于QQP和MRPC, 报告F1分数; 对于STS-B, 报告Spearman相关性; 对于其他任务, 报告准确率分数。我们排除了使用BERT作为其组件的条目。

我们使用32的批处理大小, 并在所有GLUE任务的数据上进行3个epoch的微调。对于每个任务, 我们在Dev集上选择了最佳微调学习率 (在5e-5、4e-5、3e-5和2e-5之间)。此外, 对于BERT_{LARGE}, 我们发现微调在小数据集上有时不稳定, 因此我们运行了多次随机重启, 并在Dev集上选择了最佳模型。使用随机重启时, 我们使用相同的预训练检查点, 但执行不同的微调数据洗牌和分类器层初始化。⁹

结果展示在表1中。BERT_{BASE} 和BERT_{LARGE} 在所有任务上都大幅优于所有系统, 分别获得比现有最佳技术平均准确率提高4.5%和7.0%。请注意, BERT_{BASE} 和OpenAI GPT在模型架构方面几乎相同, 除了注意力掩码。对于最大且最广泛报告的GLUE任务MNLI, BERT获得绝对准确率提高4.6%。在官方GLUE排行榜¹⁰上, BERT_{LARGE} 获得80.5分, 而截至编写日期, OpenAI GPT获得72.8分。

我们发现 BERT_{LARGE} 在所有任务中都显著优于 BERT_{BASE}, 尤其是在训练数据非常少的情况下。模型大小的效果在 5.2 节中进行了更深入的探讨。

4.2 SQuAD v1.1

斯坦福问答数据集 (SQuAD v1.1) 是一个包含 100k 个众包-来源问题/答案对(Rajpurkar 等人, 2016)。给定一个问题以及来自

⁹GLUE 数据集的分布不包含测试标签, 我们仅对 BERT_{BASE} 和 BERT_{LARGE} 各提交了一次 GLUE 评估服务器。¹⁰<https://gluebenchmark.com/leaderboard>

维基百科的段落, 其中包含答案, 任务是预测段落中的答案文本跨度。

如图 1 所示, 在问答任务中, 我们将输入问题和段落表示为单个打包序列, 问题使用 A 嵌入, 段落使用 B 嵌入。我们在微调期间仅引入一个起始向量 $S \in \mathbb{R}^H$ 和一个结束向量 $E \in \mathbb{R}^H$ 。单词 i 作为答案跨度起始的概率计算为 T_i 和 S 之间的点积, 然后对所有段落中的单词进行 softmax 操作: $P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$ 。

类似地, 用于答案跨度结束的公式相同。从位置 i 到位置 j 的候选跨度的得分定义为 $S \cdot T_i + E \cdot T_j$, 其中使用 $j \geq i$ 作为预测的最大得分跨度。训练目标是正确起始和结束位置的似然之和。我们使用学习率为 5e-5 和批大小为 32 进行 3 个 epoch 的微调。

表2显示了排行榜前几名的条目以及来自顶级发布系统 (Seo等人, 2017年; Clark和Gardner, 2018年; Peters等人, 2018a; Hu等人, 2018年) 的结果。SQuAD排行榜的前几名结果没有最新的公共系统描述可用,¹¹ 并且允许在训练其系统时使用任何公共数据。因此, 我们通过首先在TriviaQA (Joshi等人, 2017年) 上进行微调, 然后在SQuAD上进行微调, 在我们的系统中使用适度的数据增强。

我们的最佳系统在集成方面优于排行榜系统 +1.5 F1, 作为单个系统则达到+1.3 F1。事实上, 我们的单个BERT模型在F1分数方面优于排行榜系统。如果没有TriviaQA微调数据, 我们仅损失0.1-0.4 F1, 仍然以较大优势优于所有现有系统 +1

¹¹QANet在Yu等人 (2018年) 中进行了描述, 但该系统在发表后有了实质性的改进。

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} (Single)	80.8	88.5	-	-
BERT _{LARGE} (Single)	84.1	90.9	-	-
BERT _{LARGE} (Ensemble)	85.8	91.8	-	-
BERT _{LARGE} (Sgl.+TriviaQA)	84.2	91.1	85.1	91.8
BERT _{LARGE} (Ens.+TriviaQA)	86.2	92.2	87.4	93.2

Table 2: SQuAD 1.1 results. The BERT ensemble is 7x systems which use different pre-training checkpoints and fine-tuning seeds.

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-	-	71.4	74.4
Ours				
BERT _{LARGE} (Single)	78.7	81.9	80.0	83.1

Table 3: SQuAD 2.0 results. We exclude entries that use BERT as one of their components.

tuning data, we only lose 0.1-0.4 F1, still outperforming all existing systems by a wide margin.¹²

4.3 SQuAD v2.0

The SQuAD 2.0 task extends the SQuAD 1.1 problem definition by allowing for the possibility that no short answer exists in the provided paragraph, making the problem more realistic.

We use a simple approach to extend the SQuAD v1.1 BERT model for this task. We treat questions that do not have an answer as having an answer span with start and end at the [CLS] token. The probability space for the start and end answer span positions is extended to include the position of the [CLS] token. For prediction, we compare the score of the no-answer span: $s_{\text{null}} = S \cdot C + E \cdot C$ to the score of the best non-null span

¹²The TriviaQA data we used consists of paragraphs from TriviaQA-Wiki formed of the first 400 tokens in documents, that contain at least one of the provided possible answers.

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
Human (expert) [†]	-	85.0
Human (5 annotations) [†]	-	88.0

Table 4: SWAG Dev and Test accuracies. [†]Human performance is measured with 100 samples, as reported in the SWAG paper.

$s_{i,j} = \max_{j \geq i} S \cdot T_i + E \cdot T_j$. We predict a non-null answer when $s_{i,j} > s_{\text{null}} + \tau$, where the threshold τ is selected on the dev set to maximize F1. We did not use TriviaQA data for this model. We fine-tuned for 2 epochs with a learning rate of 5e-5 and a batch size of 48.

The results compared to prior leaderboard entries and top published work (Sun et al., 2018; Wang et al., 2018b) are shown in Table 3, excluding systems that use BERT as one of their components. We observe a +5.1 F1 improvement over the previous best system.

4.4 SWAG

The Situations With Adversarial Generations (SWAG) dataset contains 113k sentence-pair completion examples that evaluate grounded common-sense inference (Zellers et al., 2018). Given a sentence, the task is to choose the most plausible continuation among four choices.

When fine-tuning on the SWAG dataset, we construct four input sequences, each containing the concatenation of the given sentence (sentence A) and a possible continuation (sentence B). The only task-specific parameters introduced is a vector whose dot product with the [CLS] token representation C denotes a score for each choice which is normalized with a softmax layer.

We fine-tune the model for 3 epochs with a learning rate of 2e-5 and a batch size of 16. Results are presented in Table 4. BERT_{LARGE} outperforms the authors’ baseline ESIM+ELMo system by +27.1% and OpenAI GPT by 8.3%.

5 Ablation Studies

In this section, we perform ablation experiments over a number of facets of BERT in order to better understand their relative importance. Additional

系统	Dev		Test	
	EM	F1	EM	F1
顶级排行榜系统（2018年12月10日）				
人类	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 集成 - QANet	-	-	84.5	90.5
已发表				
BiDAF+ELMo (单模型)	-	85.6	-	85.8
R.M. 读者 (集成)	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} (Single)	80.8	88.5	-	-
BERT _{LARGE} (Single)	84.1	90.9	-	-
BERT _{LARGE} (Ensemble)	85.8	91.8	-	-
BERT _{LARGE} (Sgl.+TriviaQA)	84.2	91.1	85.1	91.8
BERT _{LARGE} (Ens. .+TriviaQA)	86.2	92.2	87.4	93.2

表2: SQuAD 1.1结果。BERT集成是7个使用不同的预训练检查点和微调种子的系统。

系统	Dev		Test	
	EM	F1	EM	F1
顶级排行榜系统（2018年12月10日）				
Human	86.3	89.0	86.9	89.5
#1 单一 - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
已发布				
unet (集成)	-	-	71.4	74.9
SLQA+ (单次)	-	-	71.4	74.4
Ours				
BERT _{LARGE} (单)	78.7	81.9	80.0	83.1

表 3: SQuAD 2.0 结果。我们排除了使用 BERT 作为其组件之一的条目。

在调整数据时，我们仅损失0.1-0.4的F1分数，仍然以很大的优势超越了所有现有系统。¹²

4.3 SQuAD v2.0

SQuAD 2.0 任务通过允许提供的段落中可能不存在短答案，扩展了 SQuAD 1.1 问题定义，使问题更加现实。

我们使用一种简单的方法来扩展此任务的 SQuAD v1.1 BERT 模型。我们将没有答案的问题视为具有从 [CLS] 标记开始和结束的答案跨度。开始和结束答案跨度位置的概率空间扩展以包含 [CLS] 标记的位置。对于预测，我们将无答案跨度的分数： $s_{\text{null}} = S \cdot C + E \cdot C$ 与最佳非空跨度的分数进行比较

¹²我们使用的 TriviaQA 数据由 TriviaQA-Wiki 形成的段落组成，这些段落包含文档中至少一个提供的可能答案的前 400 个标记。

系统	开发测试	
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
人类（专家） [†]	-	85.0
人类（5个标注） [†]	-	88.0

表4: SWAG 开发和测试的准确率。 [†]人类的表现使用100个样本进行测量，如SWAG论文中所述。

$s_{i,j} = \max_{j \geq i} S \cdot T_i + E \cdot T_j$, . 当 $s_{i,j} > s_{\text{null}} + \tau$, , 其中阈值 τ 在开发集上选择以最大化 F1 时，我们预测非空答案。我们没有使用 TriviaQA 数据来训练此模型。我们使用学习率为 5e-5 和批大小为 48 的 2 个 epoch 进行微调。

与先前的排行榜条目和顶级已发表作品（Sun 等人，2018；Wang 等人，2018b）相比的结果显示在表 3 中，排除了使用 BERT 作为其组件的系统。我们观察到在之前最佳系统上取得了 +5.1F1 的提升。

4.4 SWAG

具有对抗性生成的情境（SWAG）数据集包含 113k 个句子对补全示例，用于评估基于常识的推理（Zellers 等人，2018）。给定一个句子，任务是选择四个选项中最合理的延续。

在 SWAG 数据集上进行微调时，我们构建了四个输入序列，每个序列包含给定句子（句子 A）和一个可能的延续（句子 B）的连接。引入的唯一特定于任务的参数是一个向量，其与 [CLS] token 表示的 C 的点积表示每个选项的得分，该得分通过 softmax 层进行归一化。

我们使用学习率为2e-5和批大小为16对模型进行了3个epoch的微调。结果展示在表4中。BERT_{LARGE} 比作者的基线ESIM+ELMo系统高 +27.1%，比OpenAI GPT高8.3%。

5 消融研究

在本节中，我们对 BERT 的多个方面进行了消融实验，以更好地理解它们的相对重要性。此外

Tasks	Dev Set				
	MNLI-m (Acc)	QNLI (Acc)	MRPC (Acc)	SST-2 (Acc)	SQuAD (F1)
BERT _{BASE}	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

Table 5: Ablation over the pre-training tasks using the BERT_{BASE} architecture. “No NSP” is trained without the next sentence prediction task. “LTR & No NSP” is trained as a left-to-right LM without the next sentence prediction, like OpenAI GPT. “+ BiLSTM” adds a randomly initialized BiLSTM on top of the “LTR + No NSP” model during fine-tuning.

ablation studies can be found in Appendix C.

5.1 Effect of Pre-training Tasks

We demonstrate the importance of the deep bidirectionality of BERT by evaluating two pre-training objectives using exactly the same pre-training data, fine-tuning scheme, and hyperparameters as BERT_{BASE}:

No NSP: A bidirectional model which is trained using the “masked LM” (MLM) but without the “next sentence prediction” (NSP) task.

LTR & No NSP: A left-context-only model which is trained using a standard Left-to-Right (LTR) LM, rather than an MLM. The left-only constraint was also applied at fine-tuning, because removing it introduced a pre-train/fine-tune mismatch that degraded downstream performance. Additionally, this model was pre-trained without the NSP task. This is directly comparable to OpenAI GPT, but using our larger training dataset, our input representation, and our fine-tuning scheme.

We first examine the impact brought by the NSP task. In Table 5, we show that removing NSP hurts performance significantly on QNLI, MNLI, and SQuAD 1.1. Next, we evaluate the impact of training bidirectional representations by comparing “No NSP” to “LTR & No NSP”. The LTR model performs worse than the MLM model on all tasks, with large drops on MRPC and SQuAD.

For SQuAD it is intuitively clear that a LTR model will perform poorly at token predictions, since the token-level hidden states have no right-side context. In order to make a good faith attempt at strengthening the LTR system, we added a randomly initialized BiLSTM on top. This does significantly improve results on SQuAD, but the

results are still far worse than those of the pre-trained bidirectional models. The BiLSTM hurts performance on the GLUE tasks.

We recognize that it would also be possible to train separate LTR and RTL models and represent each token as the concatenation of the two models, as ELMo does. However: (a) this is twice as expensive as a single bidirectional model; (b) this is non-intuitive for tasks like QA, since the RTL model would not be able to condition the answer on the question; (c) this it is strictly less powerful than a deep bidirectional model, since it can use both left and right context at every layer.

5.2 Effect of Model Size

In this section, we explore the effect of model size on fine-tuning task accuracy. We trained a number of BERT models with a differing number of layers, hidden units, and attention heads, while otherwise using the same hyperparameters and training procedure as described previously.

Results on selected GLUE tasks are shown in Table 6. In this table, we report the average Dev Set accuracy from 5 random restarts of fine-tuning. We can see that larger models lead to a strict accuracy improvement across all four datasets, even for MRPC which only has 3,600 labeled training examples, and is substantially different from the pre-training tasks. It is also perhaps surprising that we are able to achieve such significant improvements on top of models which are already quite large relative to the existing literature. For example, the largest Transformer explored in Vaswani et al. (2017) is (L=6, H=1024, A=16) with 100M parameters for the encoder, and the largest Transformer we have found in the literature is (L=64, H=512, A=2) with 235M parameters (Al-Rfou et al., 2018). By contrast, BERT_{BASE} contains 110M parameters and BERT_{LARGE} contains 340M parameters.

It has long been known that increasing the model size will lead to continual improvements on large-scale tasks such as machine translation and language modeling, which is demonstrated by the LM perplexity of held-out training data shown in Table 6. However, we believe that this is the first work to demonstrate convincingly that scaling to extreme model sizes also leads to large improvements on very small scale tasks, provided that the model has been sufficiently pre-trained. Peters et al. (2018b) presented

任务	开发集				
	MNLI-m (准确率)	QNLI (准确率)	MRPC (准确率)	SST-2 (Acc)	SQuAD (F1)
BERT _{BASE}	84.4	88.4	86.7	92.7	88.5
无 NSP	83.9	84.9	86.5	92.6	87.9
LTR & 无 NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

表 5: 使用 BERT_{BASE} 架构在预训练任务上的消融实验。“无 NSP” 不包含下一句预测任务。“LTR & 无 NSP” 像 OpenAI GPT 一样, 作为不包含下一句预测的从左到右语言模型进行训练。“+ BiLSTM” 在微调期间, 在 “LTR + 无 NSP” 模型顶部添加一个随机初始化的 BiLSTM。

消融研究可以在附录 C 中找到。

5.1 预训练任务的影响

我们通过使用与BERT_{BASE}完全相同的预训练数据、微调方案和超参数来评估两个预训练目标, 以此证明BERT深度双向性的重要性:

无NSP: 一个使用“掩码语言模型”(MLM)但未使用“下一句预测”(NSP)任务的双向模型。

LTR & 无NSP: 一个仅使用标准从左到右 (LTR) 语言模型进行训练的左文模型, 而不是MLM。在微调时也应用了左文约束, 因为移除它会导致预训练/微调不匹配, 从而降低下游性能。此外, 该模型在预训练时也未使用NSP任务。这与OpenAI GPT直接可比, 但使用了我们更大的训练数据集、我们的输入表示和我们的微调方案。

我们首先检查NSP任务带来的影响。在表5中, 我们表明移除NSP在QNLI、MNLI和SQuAD 1.1上显著降低了性能。接下来, 我们通过比较“无NSP”与“LTR & 无NSP”来评估训练双向表示的影响。LTR模型在所有任务上的表现都劣于MLM模型, 在MRPC和SQuAD上表现大幅下降。

对于SQuAD来说, 直观地讲, 一个从左到右 (LTR) 的模型在token预测方面表现会很差, 因为token级别的隐藏状态没有右侧的上下文。为了在真诚地尝试加强LTR系统, 我们在顶部添加了一个随机初始化的BiLSTM。这显著提高了SQuAD的结果, 但

结果仍然远差于预训练的双向模型的结果。BiLSTM降低了GLUE任务的性能。

我们认识到, 也可以分别训练LTR和RTL模型, 并将每个标记表示为两个模型的连接, 就像ELMo一样。但是: (a) 这比单个双向模型贵两倍; (b) 对于像QA这样的任务来说, 这并不直观, 因为RTL模型无法根据问题来调整答案; (c) 这比深度双向模型严格弱, 因为它可以在每一层都使用左右上下文。

5.2 模型大小的影响

在本节中, 我们探讨了模型大小对微调任务准确性的影响。我们训练了一系列具有不同层数、隐藏单元和注意力头的BERT模型, 同时使用与之前描述相同的超参数和训练过程。

在表6中显示了在选定的GLUE任务上的结果。在这个表格中, 我们报告了从5次随机重启的微调中得到的平均Dev Set准确率。我们可以看到, 更大的模型在所有四个数据集上都带来了严格的准确率提升, 即使对于MRPC, 它只有3,600个标记的训练示例, 并且与预训练任务有实质性的不同。也许令人惊讶的是, 我们能够在相对于现有文献已经相当大的模型上实现如此显著的改进。例如, Vaswani等人(2017年)探索的最大Transformer是(L=6, H=1024, A=16), 编码器有100M参数, 而我们文献中找到的最大Transformer是(L=64, H=512, A=2), 有235M参数(Al-Rfou等人, 2018年)。相比之下, BERT_{BASE}包含110M参数, BERT_{LARGE}包含340M参数。

人们早已知道, 增加模型大小将有助于在机器翻译和语言建模等大规模任务上持续改进, 这一点由表6中展示的保留训练数据的语言模型困惑度所证明。然而, 我们认为这是首次有力地证明, 扩展到极端模型大小也能在非常小规模的任务上带来显著改进, 前提是模型已经进行了充分的预训练。Peters等人(2018b)展示了

mixed results on the downstream task impact of increasing the pre-trained bi-LM size from two to four layers and Melamud et al. (2016) mentioned in passing that increasing hidden dimension size from 200 to 600 helped, but increasing further to 1,000 did not bring further improvements. Both of these prior works used a feature-based approach — we hypothesize that when the model is fine-tuned directly on the downstream tasks and uses only a very small number of randomly initialized additional parameters, the task-specific models can benefit from the larger, more expressive pre-trained representations even when downstream task data is very small.

5.3 Feature-based Approach with BERT

All of the BERT results presented so far have used the fine-tuning approach, where a simple classification layer is added to the pre-trained model, and all parameters are jointly fine-tuned on a downstream task. However, the feature-based approach, where fixed features are extracted from the pre-trained model, has certain advantages. First, not all tasks can be easily represented by a Transformer encoder architecture, and therefore require a task-specific model architecture to be added. Second, there are major computational benefits to pre-compute an expensive representation of the training data once and then run many experiments with cheaper models on top of this representation.

In this section, we compare the two approaches by applying BERT to the CoNLL-2003 Named Entity Recognition (NER) task (Tjong Kim Sang and De Meulder, 2003). In the input to BERT, we use a case-preserving WordPiece model, and we include the maximal document context provided by the data. Following standard practice, we formulate this as a tagging task but do not use a CRF

Hyperparams				Dev Set Accuracy			
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2	
3	768	12	5.84	77.9	79.8	88.4	
6	768	3	5.24	80.6	82.2	90.7	
6	768	12	4.68	81.9	84.8	91.3	
12	768	12	3.99	84.4	86.7	92.9	
12	1024	16	3.54	85.7	86.9	93.3	
24	1024	16	3.23	86.6	87.8	93.7	

Table 6: Ablation over BERT model size. #L = the number of layers; #H = hidden size; #A = number of attention heads. “LM (ppl)” is the masked LM perplexity of held-out training data.

System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	93.1
Fine-tuning approach		
BERT _{LARGE}	96.6	92.8
BERT _{BASE}	96.4	92.4
Feature-based approach (BERT _{BASE})		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

layer in the output. We use the representation of the first sub-token as the input to the token-level classifier over the NER label set.

To ablate the fine-tuning approach, we apply the feature-based approach by extracting the activations from one or more layers *without* fine-tuning any parameters of BERT. These contextual embeddings are used as input to a randomly initialized two-layer 768-dimensional BiLSTM before the classification layer.

Results are presented in Table 7. BERT_{LARGE} performs competitively with state-of-the-art methods. The best performing method concatenates the token representations from the top four hidden layers of the pre-trained Transformer, which is only 0.3 F1 behind fine-tuning the entire model. This demonstrates that BERT is effective for both fine-tuning and feature-based approaches.

6 Conclusion

Recent empirical improvements due to transfer learning with language models have demonstrated that rich, unsupervised pre-training is an integral part of many language understanding systems. In particular, these results enable even low-resource tasks to benefit from deep unidirectional architectures. Our major contribution is further generalizing these findings to deep *bidirectional* architectures, allowing the same pre-trained model to successfully tackle a broad set of NLP tasks.

在增加预训练双向语言模型大小从两层到四层对下游任务影响的混合结果，Melamud等人(2016)顺便提到，将隐藏维度大小从200增加到600有帮助，但增加到1,000并没有带来进一步的改进。这两项先前工作都使用了基于特征的方法——我们假设，当模型直接在下游任务上进行微调，并且只使用少量随机初始化的附加参数时，特定任务的模型可以从更大、更具表达力的预训练表示中受益，即使下游任务数据非常小。

5.3 基于特征的方法与BERT

迄今为止所有展示的BERT结果都使用了微调方法，其中在预训练模型中添加了一个简单的分类层，并且所有参数都在下游任务上进行联合微调。然而，从预训练模型中提取固定特征的特征方法具有某些优势。首先，并非所有任务都能轻松地用Transformer编码器架构表示，因此需要添加特定于任务的模型架构。其次，预先计算一次昂贵的训练数据表示具有重大的计算优势，然后可以在该表示之上运行许多基于更便宜模型的实验。

在本节中，我们将BERT应用于CoNLL-2003命名实体识别（NER）任务（Tjong Kim Sang和De Meulder, 2003），并通过比较两种方法。在BERT的输入中，我们使用保留大小写的WordPiece模型，并包含数据提供的最大文档上下文。遵循标准做法，我们将此问题表述为一个标记任务，但未使用CRF层

超参数				开发集准确率			
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2	
3	768	12	5.84	77.9	79.8	88.4	
6	768	3	5.24	80.6	82.2	90.7	
6	768	12	4.68	81.9	84.8	91.3	
12	768	12	3.99	84.4	86.7	92.9	
12	1024	16	3.54	85.7	86.9	93.3	
24	1024	16	3.23	86.6	87.8	93.7	

表 6：BERT 模型大小的消融。#L = 层数；#H = 隐藏大小；#A = 注意力头的数量。“LM (ppl)”是保留训练数据的掩码 LM 难度。

系统	Dev F1	Test F1
ELMo (Peters等人, 2018a)	95.7	92.2
CVT (Clark 等人, 2018)	-	92.6
CSE (Akbik 等人, 2018)	-	93.1
微调方法		
BERT _{LARGE}	96.6	92.8
BERT _{BASE}	96.4	92.4
基于特征的 (BERT _{BASE})		
嵌入	91.0	-
倒数第二个隐藏层	95.6	-
最后一个隐藏层	94.9	-
最后四个隐藏层的加权求和	95.9	-
连接最后四个隐藏层	96.1	-
所有12层的加权求和	95.5	-

表7：CoNLL-2003命名实体识别结果。超参数使用开发集选择。报告的开发集和测试集分数是使用这些超参数在5次随机重启下平均计算的。

在输出中不使用CRF层。我们使用第一个子标记的表示作为NER标签集上token级分类器的输入。

为了消除微调方法，我们应用基于特征的方法，通过从一层或多层提取激活，而不微调BERT的任何参数。这些上下文嵌入被用作分类层之前随机初始化的768维双向LSTM的输入。

结果如表7所示。BERT_{LARGE}与最先进的方法具有竞争力。表现最好的方法连接了预训练Transformer顶部四个隐藏层的标记表示，这仅比微调整个模型落后0.3 F1。这表明BERT对于微调和基于特征的方法都有效。

6 结论

近期由于使用语言模型进行迁移学习而取得的实证改进表明，丰富的无监督预训练是许多语言理解系统的关键部分。特别是，这些结果使低资源任务也能受益于深度单向架构。我们的主要贡献是这些发现推广到深度双向架构，允许相同的预训练模型成功地处理广泛的NLP任务。

References

- Alan Akbik, Duncan Blythe, and Roland Vollgraf. 2018. Contextual string embeddings for sequence labeling. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 1638–1649.
- Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. 2018. Character-level language modeling with deeper self-attention. *arXiv preprint arXiv:1808.04444*.
- Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(Nov):1817–1853.
- Luisa Bentivogli, Bernardo Magnini, Ido Dagan, Hoa Trang Dang, and Danilo Giampiccolo. 2009. The fifth PASCAL recognizing textual entailment challenge. In *TAC. NIST*.
- John Blitzer, Ryan McDonald, and Fernando Pereira. 2006. Domain adaptation with structural correspondence learning. In *Proceedings of the 2006 conference on empirical methods in natural language processing*, pages 120–128. Association for Computational Linguistics.
- Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *EMNLP*. Association for Computational Linguistics.
- Peter F Brown, Peter V Desouza, Robert L Mercer, Vincent J Della Pietra, and Jenifer C Lai. 1992. Class-based n-gram models of natural language. *Computational linguistics*, 18(4):467–479.
- Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. 2017. [Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation](#). In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.
- Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Philipp Koehn, and Tony Robinson. 2013. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*.
- Z. Chen, H. Zhang, X. Zhang, and L. Zhao. 2018. [Quora question pairs](#).
- Christopher Clark and Matt Gardner. 2018. Simple and effective multi-paragraph reading comprehension. In *ACL*.

- Kevin Clark, Minh-Thang Luong, Christopher D Manning, and Quoc Le. 2018. Semi-supervised sequence modeling with cross-view training. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1914–1925.
- Ronan Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*, pages 160–167. ACM.
- Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. 2017. [Supervised learning of universal sentence representations from natural language inference data](#). In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, Copenhagen, Denmark. Association for Computational Linguistics.
- Andrew M Dai and Quoc V Le. 2015. Semi-supervised sequence learning. In *Advances in neural information processing systems*, pages 3079–3087.
- J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*.
- William B Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*.
- William Fedus, Ian Goodfellow, and Andrew M Dai. 2018. Maskgan: Better text generation via filling in the... *arXiv preprint arXiv:1801.07736*.
- Dan Hendrycks and Kevin Gimpel. 2016. [Bridging nonlinearities and stochastic regularizers with gaussian error linear units](#). *CoRR*, abs/1606.08415.
- Felix Hill, Kyunghyun Cho, and Anna Korhonen. 2016. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics.
- Jeremy Howard and Sebastian Ruder. 2018. [Universal language model fine-tuning for text classification](#). In *ACL*. Association for Computational Linguistics.
- Minghao Hu, Yuxing Peng, Zhen Huang, Xipeng Qiu, Furu Wei, and Ming Zhou. 2018. Reinforced mnemonic reader for machine reading comprehension. In *IJCAI*.
- Yacine Jernite, Samuel R. Bowman, and David Sontag. 2017. [Discourse-based objectives for fast unsupervised sentence representation learning](#). *CoRR*, abs/1705.00557.

参考文献

- Alan Akbik, Duncan Blythe, and Roland Vollgraf. 2018. Contextual string embeddings for sequence labeling. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 1638–1649.
- Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. 2018. Character-level language modeling with deeper self-attention. *arXiv preprint arXiv:1808.04444*.
- Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(Nov):1817–1853.
- Luisa Bentivogli, Bernardo Magnini, Ido Dagan, Hoa Trang Dang, and Danilo Giampiccolo. 2009. The fifth PASCAL recognizing textual entailment challenge. In *TAC. NIST*.
- John Blitzer, Ryan McDonald, and Fernando Pereira. 2006. Domain adaptation with structural correspondence learning. In *Proceedings of the 2006 conference on empirical methods in natural language processing*, pages 120–128. Association for Computational Linguistics.
- Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *EMNLP*. Association for Computational Linguistics.
- Peter F Brown, Peter V Desouza, Robert L Mercer, Vincent J Della Pietra, and Jenifer C Lai. 1992. Class-based n-gram models of natural language. *Computational linguistics*, 18(4):467–479.
- Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. 2017. [Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation](#). In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.
- Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Philipp Koehn, and Tony Robinson. 2013. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*.
- Z. Chen, H. Zhang, X. Zhang, and L. Zhao. 2018. [Quora question pairs](#).
- Christopher Clark and Matt Gardner. 2018. Simple and effective multi-paragraph reading comprehension. In *ACL*.

- Kevin Clark, Minh-Thang Luong, Christopher D Manning, and Quoc Le. 2018. Semi-supervised sequence modeling with cross-view training. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1914–1925.
- Ronan Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*, pages 160–167. ACM.
- Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. 2017. [Supervised learning of universal sentence representations from natural language inference data](#). In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, Copenhagen, Denmark. Association for Computational Linguistics.
- Andrew M Dai and Quoc V Le. 2015. Semi-supervised sequence learning. In *Advances in neural information processing systems*, pages 3079–3087.
- J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*.
- William B Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*.
- William Fedus, Ian Goodfellow, and Andrew M Dai. 2018. Maskgan: Better text generation via filling in the... *arXiv preprint arXiv:1801.07736*.
- Dan Hendrycks and Kevin Gimpel. 2016. [Bridging nonlinearities and stochastic regularizers with gaussian error linear units](#). *CoRR*, abs/1606.08415.
- Felix Hill, Kyunghyun Cho, and Anna Korhonen. 2016. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics.
- Jeremy Howard and Sebastian Ruder. 2018. [Universal language model fine-tuning for text classification](#). In *ACL*. Association for Computational Linguistics.
- Minghao Hu, Yuxing Peng, Zhen Huang, Xipeng Qiu, Furu Wei, and Ming Zhou. 2018. Reinforced mnemonic reader for machine reading comprehension. In *IJCAI*.
- Yacine Jernite, Samuel R. Bowman, and David Sontag. 2017. [Discourse-based objectives for fast unsupervised sentence representation learning](#). *CoRR*, abs/1705.00557.

- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *ACL*.
- Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In *Advances in neural information processing systems*, pages 3294–3302.
- Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International Conference on Machine Learning*, pages 1188–1196.
- Hector J Levesque, Ernest Davis, and Leora Morgenstern. 2011. The winograd schema challenge. In *Aaai spring symposium: Logical formalizations of commonsense reasoning*, volume 46, page 47.
- Lajanugen Logeswaran and Honglak Lee. 2018. [An efficient framework for learning sentence representations](#). In *International Conference on Learning Representations*.
- Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. 2017. Learned in translation: Contextualized word vectors. In *NIPS*.
- Oren Melamud, Jacob Goldberger, and Ido Dagan. 2016. context2vec: Learning generic context embedding with bidirectional LSTM. In *CoNLL*.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc.
- Andriy Mnih and Geoffrey E Hinton. 2009. [A scalable hierarchical distributed language model](#). In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems 21*, pages 1081–1088. Curran Associates, Inc.
- Ankur P Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. 2016. A decomposable attention model for natural language inference. In *EMNLP*.
- Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. [Glove: Global vectors for word representation](#). In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.
- Matthew Peters, Waleed Ammar, Chandra Bhagavata, and Russell Power. 2017. Semi-supervised sequence tagging with bidirectional language models. In *ACL*.
- Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018a. Deep contextualized word representations. In *NAACL*.
- Matthew Peters, Mark Neumann, Luke Zettlemoyer, and Wen-tau Yih. 2018b. Dissecting contextual word embeddings: Architecture and representation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1499–1509.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding with unsupervised learning. Technical report, OpenAI.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.
- Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. 2017. Bidirectional attention flow for machine comprehension. In *ICLR*.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.
- Fu Sun, Linyang Li, Xipeng Qiu, and Yang Liu. 2018. U-net: Machine reading comprehension with unanswerable questions. *arXiv preprint arXiv:1810.06638*.
- Wilson L Taylor. 1953. Cloze procedure: A new tool for measuring readability. *Journalism Bulletin*, 30(4):415–433.
- Erik F Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition. In *CoNLL*.
- Joseph Turian, Lev Ratinov, and Yoshua Bengio. 2010. Word representations: A simple and general method for semi-supervised learning. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, ACL '10*, pages 384–394.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 6000–6010.
- Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103. ACM.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018a. Glue: A multi-task benchmark and analysis platform
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *ACL*.
- Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In *Advances in neural information processing systems*, pages 3294–3302.
- Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International Conference on Machine Learning*, pages 1188–1196.
- Hector J Levesque, Ernest Davis, and Leora Morgenstern. 2011. The winograd schema challenge. In *Aaai spring symposium: Logical formalizations of commonsense reasoning*, volume 46, page 47.
- Lajanugen Logeswaran and Honglak Lee. 2018. [An efficient framework for learning sentence representations](#). In *International Conference on Learning Representations*.
- Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. 2017. Learned in translation: Contextualized word vectors. In *NIPS*.
- Oren Melamud, Jacob Goldberger, and Ido Dagan. 2016. context2vec: Learning generic context embedding with bidirectional LSTM. In *CoNLL*.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc.
- Andriy Mnih and Geoffrey E Hinton. 2009. [A scalable hierarchical distributed language model](#). In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems 21*, pages 1081–1088. Curran Associates, Inc.
- Ankur P Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. 2016. A decomposable attention model for natural language inference. In *EMNLP*.
- Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. [Glove: Global vectors for word representation](#). In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.
- Matthew Peters, Waleed Ammar, Chandra Bhagavata, and Russell Power. 2017. Semi-supervised sequence tagging with bidirectional language models. In *ACL*.
- Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018a. Deep contextualized word representations. In *NAACL*.
- Matthew Peters, Mark Neumann, Luke Zettlemoyer, and Wen-tau Yih. 2018b. Dissecting contextual word embeddings: Architecture and representation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1499–1509.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding with unsupervised learning. Technical report, OpenAI.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.
- Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. 2017. Bidirectional attention flow for machine comprehension. In *ICLR*.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.
- Fu Sun, Linyang Li, Xipeng Qiu, and Yang Liu. 2018. U-net: Machine reading comprehension with unanswerable questions. *arXiv preprint arXiv:1810.06638*.
- Wilson L Taylor. 1953. Cloze procedure: A new tool for measuring readability. *Journalism Bulletin*, 30(4):415–433.
- Erik F Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition. In *CoNLL*.
- Joseph Turian, Lev Ratinov, and Yoshua Bengio. 2010. Word representations: A simple and general method for semi-supervised learning. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, ACL '10*, pages 384–394.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 6000–6010.
- Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103. ACM.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018a. Glue: A multi-task benchmark and analysis platform

for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355.

Wei Wang, Ming Yan, and Chen Wu. 2018b. Multi-granularity hierarchical attention fusion networks for reading comprehension and question answering. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2018. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*.

Adina Williams, Nikita Nangia, and Samuel R Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL*.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. How transferable are features in deep neural networks? In *Advances in neural information processing systems*, pages 3320–3328.

Adams Wei Yu, David Dohan, Minh-Thang Luong, Rui Zhao, Kai Chen, Mohammad Norouzi, and Quoc V Le. 2018. QANet: Combining local convolution with global self-attention for reading comprehension. In *ICLR*.

Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. 2018. Swag: A large-scale adversarial dataset for grounded commonsense inference. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27.

Appendix for “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”

We organize the appendix into three sections:

- Additional implementation details for BERT are presented in Appendix A;

- Additional details for our experiments are presented in Appendix B; and

- Additional ablation studies are presented in Appendix C.

We present additional ablation studies for BERT including:

- Effect of Number of Training Steps; and
- Ablation for Different Masking Procedures.

A Additional Details for BERT

A.1 Illustration of the Pre-training Tasks

We provide examples of the pre-training tasks in the following.

Masked LM and the Masking Procedure Assuming the unlabeled sentence is `my dog is hairy`, and during the random masking procedure we chose the 4-th token (which corresponding to `hairy`), our masking procedure can be further illustrated by

- 80% of the time: Replace the word with the [MASK] token, e.g., `my dog is hairy` → `my dog is [MASK]`
- 10% of the time: Replace the word with a random word, e.g., `my dog is hairy` → `my dog is apple`
- 10% of the time: Keep the word unchanged, e.g., `my dog is hairy` → `my dog is hairy`. The purpose of this is to bias the representation towards the actual observed word.

The advantage of this procedure is that the Transformer encoder does not know which words it will be asked to predict or which have been replaced by random words, so it is forced to keep a distributional contextual representation of every input token. Additionally, because random replacement only occurs for 1.5% of all tokens (i.e., 10% of 15%), this does not seem to harm the model’s language understanding capability. In Section C.2, we evaluate the impact this procedure.

Compared to standard language model training, the masked LM only make predictions on 15% of tokens in each batch, which suggests that more pre-training steps may be required for the model

for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355.

Wei Wang, Ming Yan, and Chen Wu. 2018b. Multi-granularity hierarchical attention fusion networks for reading comprehension and question answering. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2018. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*.

Adina Williams, Nikita Nangia, and Samuel R Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL*.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. How transferable are features in deep neural networks? In *Advances in neural information processing systems*, pages 3320–3328.

Adams Wei Yu, David Dohan, Minh-Thang Luong, Rui Zhao, Kai Chen, Mohammad Norouzi, and Quoc V Le. 2018. QANet: Combining local convolution with global self-attention for reading comprehension. In *ICLR*.

Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. 2018. Swag: A large-scale adversarial dataset for grounded commonsense inference. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27.

“BERT: 深度双向Transformer语言理解预训练”的附录

我们将附录分为三个部分:

- BERT的附加实现细节在附录A中呈现;

- 我们实验的更多细节是在附录B中呈现; 和

- 额外的消融研究在附录C中呈现。

我们展示了额外的消融研究, 包括BERT, 包括:

- 训练步数的影响; 以及
- 不同掩码过程的消融程序。

A BERT的附加详细信息

A.1 预训练任务说明

我们将在以下提供预训练任务的示例。

Masked LM and the Masking Procedure Assuming the unlabeled sentence is `my dog is hairy`, and during the random masking procedure we chose the 4-th token (which corresponding to `hairy`), our masking procedure can be further illustrated by

- 80% 的情况下: 用 [MASK] 标记替换单词, 例如, 我的狗毛茸茸的 → 我的狗毛茸茸的 [MASK]
- 10% 的情况下: 用随机单词替换单词, 例如, 我的狗毛茸茸的 → 我的狗毛茸茸的苹果
- 10% 的情况下: 保持单词不变, 例如, 我的狗毛茸茸的 → 我的狗毛茸茸的。这样做的目的是使表示偏向实际观察到的单词。

此程序的优点在于 Transformer encoder 不知道它将被要求预测哪些单词或哪些已经被重新-由随机词替换, 因此它必须保持每个输入标记的分布式上下文表示。此外, 因为随机替换仅发生在所有标记的1.5% (即15%的10%), 这似乎不会造成伤害

the model’s language understanding capability. In Section C.2, we evaluate the impact this procedure.

与标准语言模型训练相比, 掩码语言模型每次只在每个批次的15%的token上进行预测, 这表明模型可能需要更多的预训练步骤才能收敛

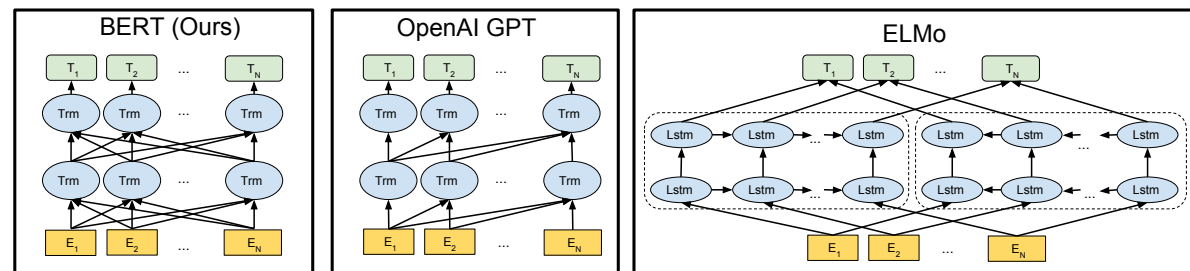


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

to converge. In Section C.1 we demonstrate that MLM does converge marginally slower than a left-to-right model (which predicts every token), but the empirical improvements of the MLM model far outweigh the increased training cost.

Next Sentence Prediction The next sentence prediction task can be illustrated in the following examples.

Input = [CLS] the man went to [MASK] store [SEP]
he bought a gallon [MASK] milk [SEP]
Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]
penguin [MASK] are flight ##less birds [SEP]
Label = NotNext

A.2 Pre-training Procedure

To generate each training input sequence, we sample two spans of text from the corpus, which we refer to as “sentences” even though they are typically much longer than single sentences (but can be shorter also). The first sentence receives the A embedding and the second receives the B embedding. 50% of the time B is the actual next sentence that follows A and 50% of the time it is a random sentence, which is done for the “next sentence prediction” task. They are sampled such that the combined length is ≤ 512 tokens. The LM masking is applied after WordPiece tokenization with a uniform masking rate of 15%, and no special consideration given to partial word pieces.

We train with batch size of 256 sequences (256 sequences * 512 tokens = 128,000 tokens/batch) for 1,000,000 steps, which is approximately 40

epochs over the 3.3 billion word corpus. We use Adam with learning rate of $1e-4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, L2 weight decay of 0.01, learning rate warmup over the first 10,000 steps, and linear decay of the learning rate. We use a dropout probability of 0.1 on all layers. We use a gelu activation (Hendrycks and Gimpel, 2016) rather than the standard relu, following OpenAI GPT. The training loss is the sum of the mean masked LM likelihood and the mean next sentence prediction likelihood.

Training of BERT_{BASE} was performed on 4 Cloud TPUs in Pod configuration (16 TPU chips total).¹³ Training of BERT_{LARGE} was performed on 16 Cloud TPUs (64 TPU chips total). Each pre-training took 4 days to complete.

Longer sequences are disproportionately expensive because attention is quadratic to the sequence length. To speed up pretraining in our experiments, we pre-train the model with sequence length of 128 for 90% of the steps. Then, we train the rest 10% of the steps of sequence of 512 to learn the positional embeddings.

A.3 Fine-tuning Procedure

For fine-tuning, most model hyperparameters are the same as in pre-training, with the exception of the batch size, learning rate, and number of training epochs. The dropout probability was always kept at 0.1. The optimal hyperparameter values are task-specific, but we found the following range of possible values to work well across all tasks:

- Batch size: 16, 32

¹³<https://cloudplatform.googleblog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html>

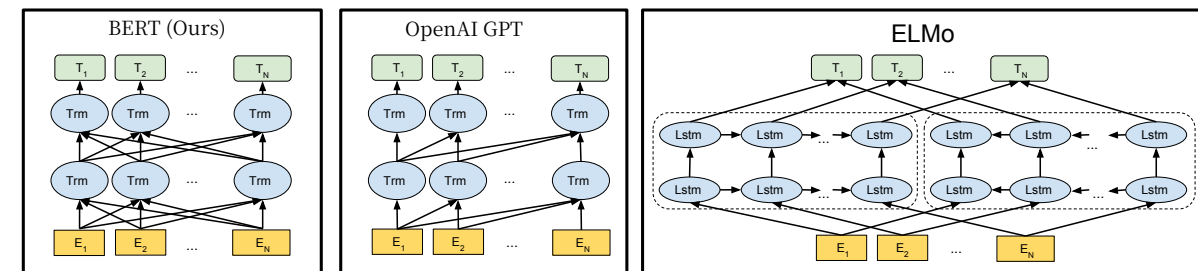


图3：预训练模型架构的差异。BERT使用双向Transformer。OpenAI GPT使用从左到右的Transformer。ELMo使用独立训练的从左到右和从右到左的LSTM的连接来生成下游任务的特征。在三者中，只有BERT表示在所有层中都联合依赖于左右上下文。除了架构差异外，BERT和OpenAI GPT是微调方法，而ELMo是基于特征的方法。

收敛。在C.1节中，我们证明了MLM的收敛速度边际上比从左到右的模型（预测每个token）慢，但MLM模型的实证改进远远超过了增加的训练成本。

句子预测 The next sentence prediction task can be illustrated in the following examples.

输入 = [CLS] 那人去了 [MASK] 商店 [SEP]
他买了一加仑 [MASK] 牛奶 [SEP]
标签 = IsNext

输入 = [CLS] 男人 [MASK] 到商店 [SEP]
企鹅 [MASK] 是飞行 ##无能的鸟类 [SEP]
标签 = NotNext

A.2 预训练步骤

为了生成每个训练输入序列，我们从语料库中采样两个文本片段，我们称之为“句子”，尽管它们通常比单个句子长得多（但也可能更短）。第一个句子接收 A 嵌入，第二个句子接收 B 嵌入。50% 的时间里 B 是紧随 A 的实际下一句，50% 的时间里它是一个随机句子，这是为了“下一句预测”任务。它们被采样，使得总长度为 ≤ 512 个 token。LM 掩码在 WordPiece 分词后应用，掩码率为 15%，对部分词片段不做特殊处理。

我们使用256序列（256序列 * 512个token = 128, 每批1000个token）进行训练，共1,000,000步，大约需要40

在3.3亿词语料库上进行多个epoch的训练。我们使用Adam优化器，学习率为 $1e-4$ ， $\beta_1 = 0.9, \beta_2 = 0.999$ ，L2权重衰减为0.01，前10,000步进行学习率预热，并采用学习率的线性衰减。所有层都使用0.1的dropout概率。我们使用GELU激活函数（Hendrycks和Gimpel, 2016），而不是标准的ReLU，遵循OpenAI GPT的做法。训练损失是平均掩码语言模型似然和平均下一句预测似然的和。

BERT_{BASE} 的训练在 4 个 Cloud TPU 上以 Pod 配置（总共 16 个 TPU 芯片）进行。¹³ BERT_{LARGE} 的训练在 16 个 Cloud TPU 上进行（总共 64 个 TPU 芯片）。每个预训练需要 4 天完成。

较长的序列成本不成比例地高，因为注意力机制与序列长度呈平方关系。为了加速我们实验中的预训练，我们用128长度的序列对模型进行90%步骤的预训练。然后，我们用512长度的序列训练剩余的10%步骤，以学习位置嵌入。

A.3 微调流程

对于微调，大多数模型超参数与预训练时相同，例外的是批大小、学习率和训练轮数。丢弃率始终保持在0.1。最佳超参数值是任务特定的，但我们发现以下范围内的可能值在所有任务中都能很好地工作：

- 批大小: 16, 32

¹³<https://cloudplatform.googleblog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html>

- **Learning rate (Adam):** 5e-5, 3e-5, 2e-5
- **Number of epochs:** 2, 3, 4

We also observed that large data sets (e.g., 100k+ labeled training examples) were far less sensitive to hyperparameter choice than small data sets. Fine-tuning is typically very fast, so it is reasonable to simply run an exhaustive search over the above parameters and choose the model that performs best on the development set.

A.4 Comparison of BERT, ELMo, and OpenAI GPT

Here we study the differences in recent popular representation learning models including ELMo, OpenAI GPT and BERT. The comparisons between the model architectures are shown visually in Figure 3. Note that in addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

The most comparable existing pre-training method to BERT is OpenAI GPT, which trains a left-to-right Transformer LM on a large text corpus. In fact, many of the design decisions in BERT were intentionally made to make it as close to GPT as possible so that the two methods could be minimally compared. The core argument of this work is that the bi-directionality and the two pre-training tasks presented in Section 3.1 account for the majority of the empirical improvements, but we do note that there are several other differences between how BERT and GPT were trained:

- GPT is trained on the BooksCorpus (800M words); BERT is trained on the BooksCorpus (800M words) and Wikipedia (2,500M words).
- GPT uses a sentence separator ([SEP]) and classifier token ([CLS]) which are only introduced at fine-tuning time; BERT learns [SEP], [CLS] and sentence A/B embeddings during pre-training.
- GPT was trained for 1M steps with a batch size of 32,000 words; BERT was trained for 1M steps with a batch size of 128,000 words.
- GPT used the same learning rate of 5e-5 for all fine-tuning experiments; BERT chooses a task-specific fine-tuning learning rate which performs the best on the development set.

To isolate the effect of these differences, we perform ablation experiments in Section 5.1 which demonstrate that the majority of the improvements are in fact coming from the two pre-training tasks and the bidirectionality they enable.

A.5 Illustrations of Fine-tuning on Different Tasks

The illustration of fine-tuning BERT on different tasks can be seen in Figure 4. Our task-specific models are formed by incorporating BERT with one additional output layer, so a minimal number of parameters need to be learned from scratch. Among the tasks, (a) and (b) are sequence-level tasks while (c) and (d) are token-level tasks. In the figure, E represents the input embedding, T_i represents the contextual representation of token i , [CLS] is the special symbol for classification output, and [SEP] is the special symbol to separate non-consecutive token sequences.

B Detailed Experimental Setup

B.1 Detailed Descriptions for the GLUE Benchmark Experiments.

Our GLUE results in Table 1 are obtained from <https://gluebenchmark.com/leaderboard> and <https://blog.openai.com/language-unsupervised>. The GLUE benchmark includes the following datasets, the descriptions of which were originally summarized in Wang et al. (2018a):

MNLI Multi-Genre Natural Language Inference is a large-scale, crowdsourced entailment classification task (Williams et al., 2018). Given a pair of sentences, the goal is to predict whether the second sentence is an *entailment*, *contradiction*, or *neutral* with respect to the first one.

QQP Quora Question Pairs is a binary classification task where the goal is to determine if two questions asked on Quora are semantically equivalent (Chen et al., 2018).

QNLI Question Natural Language Inference is a version of the Stanford Question Answering Dataset (Rajpurkar et al., 2016) which has been converted to a binary classification task (Wang et al., 2018a). The positive examples are (question, sentence) pairs which do contain the correct answer, and the negative examples are (question, sentence) from the same paragraph which do not contain the answer.

- **学习率 (Adam) :** 5e-5, 3e-5, 2e-5
- **训练轮数:** 2, 3, 4

我们还观察到, 大型数据集 (例如, 100k+ 标记的训练示例) 对超参数选择远不如小型数据集敏感。微调通常非常快, 因此简单地在上面的参数上运行穷举搜索, 并选择在开发集上表现最好的模型是合理的。

A.4 BERT、ELMo和OpenAI GPT的比较

在这里, 我们研究了最近流行的表示学习模型 (包括ELMo、OpenAI GPT和BERT) 之间的差异。模型架构的比较在图3中直观地展示出来。请注意, 除了架构差异之外, BERT和OpenAI GPT是微调方法, 而ELMo是一种基于特征的方法。

BERT最可比的现有预训练方法是 OpenAI GPT, 它在大型文本语料库上训练一个从左到右的Transformer LM。事实上, BERT中的许多设计决策都是有意使其尽可能接近GPT, 以便这两种方法可以最小程度地进行比较。这项工作的核心论点是, 第3.1节中提出的双向性和两个预训练任务解释了大多数经验改进, 但我们确实注意到, BERT和GPT在训练方式上存在其他几个差异:

- GPT在BooksCorpus (8亿词) 上训练; BERT在BooksCorpus (8亿词) 和维基百科 (25亿词) 上训练。
- GPT使用句子分隔符 ([SEP]) 和分类标记 ([CLS]), 这些只在微调时引入; BERT学习[SEP] [CLS]并在预训练期间嵌入句子A/B。
- GPT用32,000词的批处理大小训练了1M步; BERT用128,000词的批处理大小训练了1M步。
- GPT对所有微调实验都使用了5e-5的相同学习率; BERT选择在开发集上表现最佳的特定任务微调学习率。

为了隔离这些差异的影响, 我们在第5.1节进行消融实验, 这些实验表明, 事实上大多数改进确实来自两个预训练任务及其带来的双向性。

A.5 不同任务上的微调示例

在图4中可以看到BERT在不同任务上的微调示例。我们的特定任务模型是通过将BERT与一个额外的输出层结合形成的, 因此只需学习最少数量的参数。在这些任务中, (a)和(b)是序列级任务, 而(c)和(d)是标记级任务。在图中, E 表示输入嵌入, T_i 表示标记 i 的上下文表示, [CLS] 是分类输出的特殊符号, 而 [SEP] 是用于分隔非连续标记序列的特殊符号。

B 详细实验设置

B.1 详细描述GLUE基准实验。

我们的GLUE结果在表1中来自 <https://gluebenchmark.com/> 排行榜和 <https://blog.openai.com/language-unsupervised>. GLUE基准包括以下数据集, 其描述最初在Wang等人 (2018a) 中总结:

MNLI 多领域自然语言推理是一个大规模的众包蕴涵分类任务 (Williams等人, 2018)。给定一对句子, 目标是预测第二个句子相对于第一个句子是蕴涵、矛盾还是中立。

QQP Quora Question Pairs 是一个二元分类任务, 目标是确定在 Quora 上提出的两个问题在语义上是否等价 (Chen 等人, 2018 年)。

QNLI 问题自然语言推理是斯坦福问答数据集 (Rajpurkar 等人, 2016 年) 的一个版本, 该数据集已被转换为二元分类任务 (Wang 等人, 2018a 年)。阳性示例是包含正确答案的 (问题, 句子) 对, 而阴性示例是来自同一段落的 (问题, 句子), 其中不包含答案。

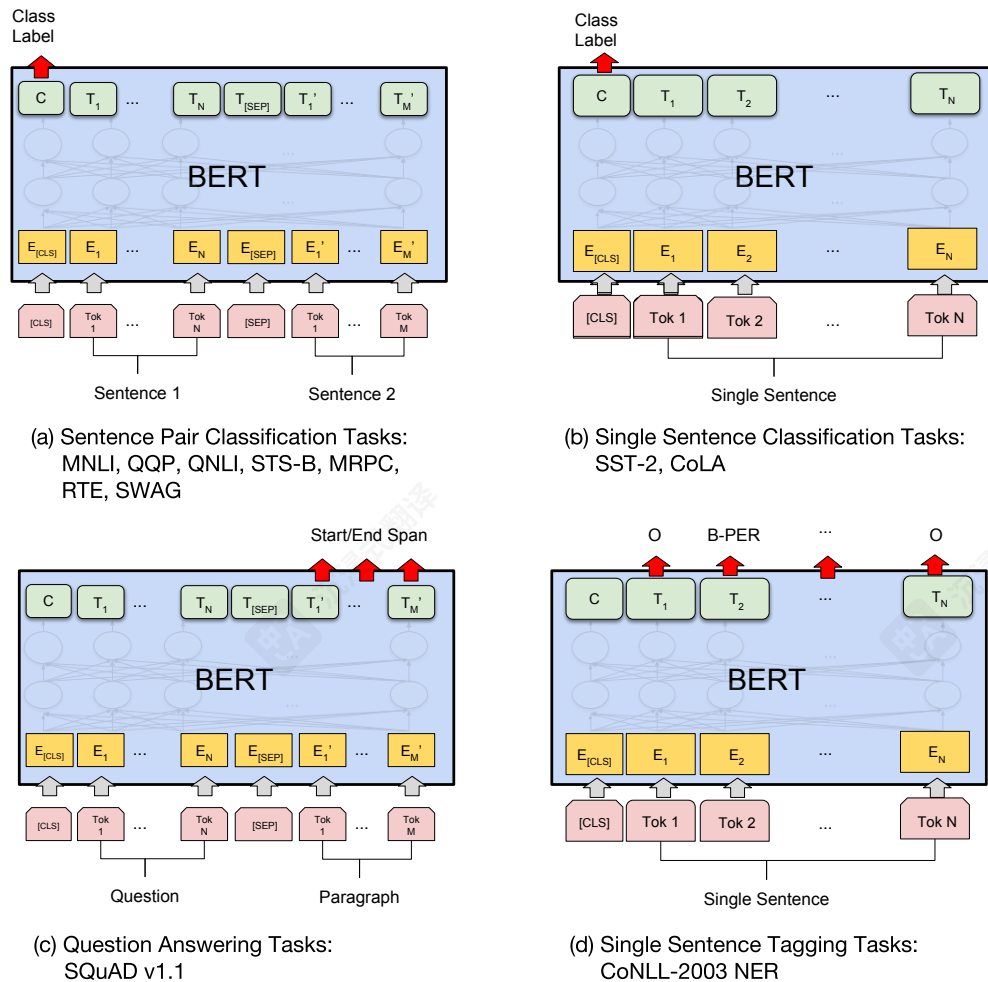


Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

SST-2 The Stanford Sentiment Treebank is a binary single-sentence classification task consisting of sentences extracted from movie reviews with human annotations of their sentiment (Socher et al., 2013).

CoLA The Corpus of Linguistic Acceptability is a binary single-sentence classification task, where the goal is to predict whether an English sentence is linguistically “acceptable” or not (Warstadt et al., 2018).

STS-B The Semantic Textual Similarity Benchmark is a collection of sentence pairs drawn from news headlines and other sources (Cer et al., 2017). They were annotated with a score from 1 to 5 denoting how similar the two sentences are in terms of semantic meaning.

MRPC Microsoft Research Paraphrase Corpus consists of sentence pairs automatically extracted from online news sources, with human annotations

for whether the sentences in the pair are semantically equivalent (Dolan and Brockett, 2005).

RTE Recognizing Textual Entailment is a binary entailment task similar to MNLI, but with much less training data (Bentivogli et al., 2009).¹⁴

WNLI Winograd NLI is a small natural language inference dataset (Levesque et al., 2011). The GLUE webpage notes that there are issues with the construction of this dataset,¹⁵ and every trained system that’s been submitted to GLUE has performed worse than the 65.1 baseline accuracy of predicting the majority class. We therefore exclude this set to be fair to OpenAI GPT. For our GLUE submission, we always predicted the ma-

¹⁴Note that we only report single-task fine-tuning results in this paper. A multitask fine-tuning approach could potentially push the performance even further. For example, we did observe substantial improvements on RTE from multitask training with MNLI.

¹⁵<https://gluebenchmark.com/faq>

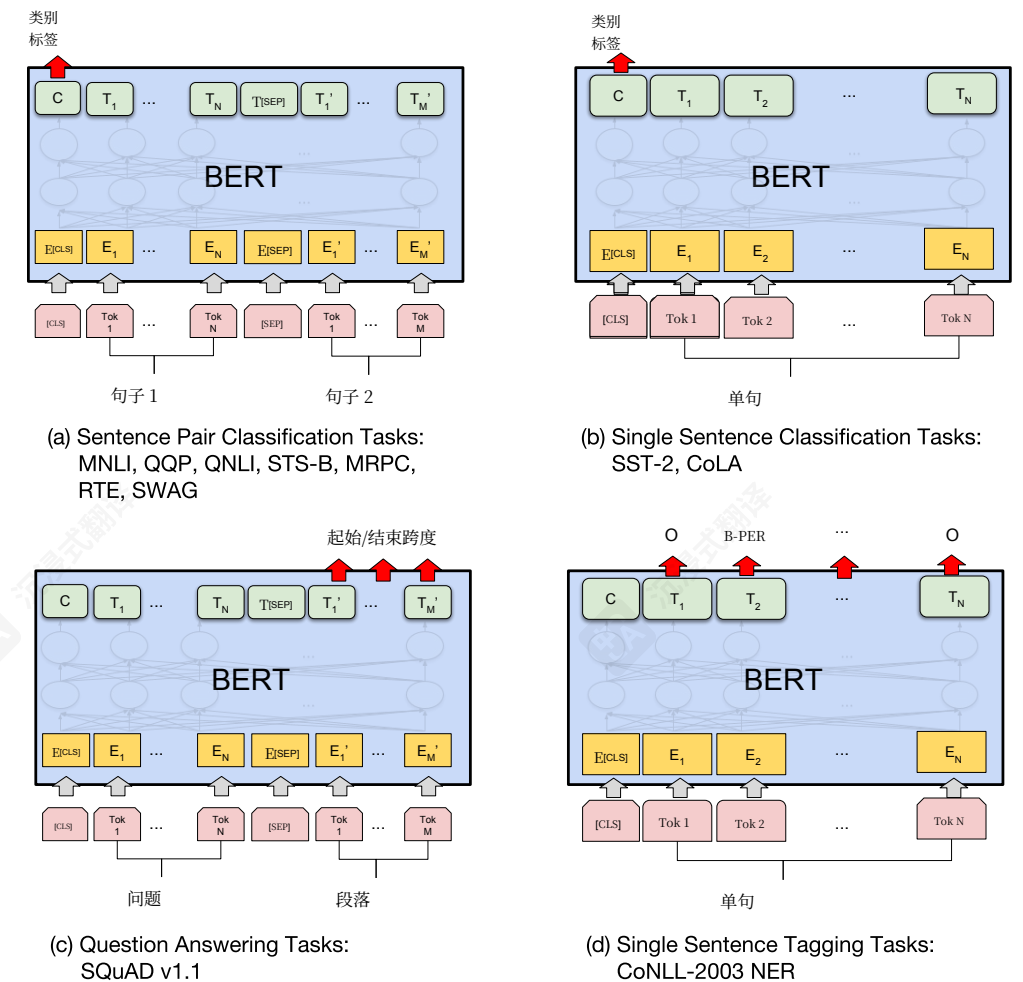


图 4：在不同任务上微调 BERT 的插图。

SST-2 斯坦福情感树库是一个二元单句分类任务，由从电影评论中提取的句子组成，这些句子带有人类对其情感的人工标注（Socher 等人，2013 年）。

CoLA The Corpus of Linguistic Acceptability 是一个二进制单句分类任务，其目标是为预测一个英语句子在语言上是否“可接受”（Warstadt et al., 2018）。

STS-B 语义文本相似度基准是一个从新闻标题和其他来源（Cer 等人，2017 年）收集的句子对集合。它们被标注了从 1 到 5 的分数，表示两个句子在语义意义上的相似程度。

MRPC Microsoft Research Paraphrase Corpus 由从在线新闻源自动提取的句子对组成，并带有人工标注

用于判断句子对中的句子是否在语义上等价（Dolan and Brockett, 2005）。

RTE 识别文本蕴涵是一个类似于 MNLI 的二元蕴涵任务，但训练数据要少得多（Bentivoglio 等人，2009 年）。¹⁴

WNLI Winograd NLI 是一个小型自然语言推理数据集（Levesque 等人，2011 年）。GLUE 网页注明该数据集的构建存在问题，¹⁵ 并且提交到 GLUE 的每个训练系统都表现不如预测多数类的 65.1 基线准确率。因此，我们将其排除以公平对待 OpenAI GPT。在我们的 GLUE 提交中，我们始终预测多数类。

¹⁴请注意，本文仅报告了单任务微调结果。多任务微调方法有可能进一步提升性能。例如，我们确实观察到在 MNLI 的多任务训练中 RTE 有显著改进。¹⁵h

<https://gluebenchmark.com/faq>

jority class.

C Additional Ablation Studies

C.1 Effect of Number of Training Steps

Figure 5 presents MNLI Dev accuracy after fine-tuning from a checkpoint that has been pre-trained for k steps. This allows us to answer the following questions:

1. Question: Does BERT really need such a large amount of pre-training (128,000 words/batch * 1,000,000 steps) to achieve high fine-tuning accuracy?

Answer: Yes, BERT_{BASE} achieves almost 1.0% additional accuracy on MNLI when trained on 1M steps compared to 500k steps.
2. Question: Does MLM pre-training converge slower than LTR pre-training, since only 15% of words are predicted in each batch rather than every word?

Answer: The MLM model does converge slightly slower than the LTR model. However, in terms of absolute accuracy the MLM model begins to outperform the LTR model almost immediately.

C.2 Ablation for Different Masking Procedures

In Section 3.1, we mention that BERT uses a mixed strategy for masking the target tokens when pre-training with the masked language model (MLM) objective. The following is an ablation study to evaluate the effect of different masking strategies.

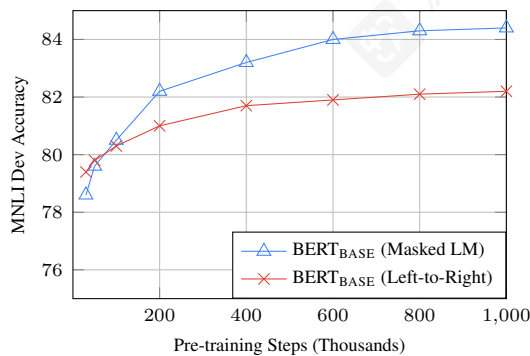


Figure 5: Ablation over number of training steps. This shows the MNLI accuracy after fine-tuning, starting from model parameters that have been pre-trained for k steps. The x-axis is the value of k .

Note that the purpose of the masking strategies is to reduce the mismatch between pre-training and fine-tuning, as the [MASK] symbol never appears during the fine-tuning stage. We report the Dev results for both MNLI and NER. For NER, we report both fine-tuning and feature-based approaches, as we expect the mismatch will be amplified for the feature-based approach as the model will not have the chance to adjust the representations.

Masking Rates			Dev Set Results		
MASK	SAME	RND	MNLI Fine-tune	NER Fine-tune	NER Feature-based
80%	10%	10%	84.2	95.4	94.9
100%	0%	0%	84.3	94.9	94.0
80%	0%	20%	84.1	95.2	94.6
80%	20%	0%	84.4	95.2	94.7
0%	20%	80%	83.7	94.8	94.6
0%	0%	100%	83.6	94.9	94.6

Table 8: Ablation over different masking strategies.

The results are presented in Table 8. In the table, MASK means that we replace the target token with the [MASK] symbol for MLM; SAME means that we keep the target token as is; RND means that we replace the target token with another random token.

The numbers in the left part of the table represent the probabilities of the specific strategies used during MLM pre-training (BERT uses 80%, 10%, 10%). The right part of the paper represents the Dev set results. For the feature-based approach, we concatenate the last 4 layers of BERT as the features, which was shown to be the best approach in Section 5.3.

From the table it can be seen that fine-tuning is surprisingly robust to different masking strategies. However, as expected, using only the MASK strategy was problematic when applying the feature-based approach to NER. Interestingly, using only the RND strategy performs much worse than our strategy as well.

多数类。

C 额外消融研究

C.1 训练步数的影响

图 5 展示了从预训练了 k 步的检查点进行微调后的 MNLI 开发集准确率。这使我们能够回答以下问题:

1. 问题: BERT 真的需要如此大量的预训练 (128,000 词/批次 * 1,000,000 步) 才能实现高微调准确率吗?

答案: 是的, BERT_{BASE} 在 1M 步训练时比 500k 步训练在 MNLI 上实现了近 1.0% 的额外准确率。
2. 问题: 由于每个批次仅预测 15% 的词而不是每个词, MLM 预训练是否比 LTR 预训练收敛更慢?

答案: MLM模型确实收敛稍微比LTR模型慢。然而, 在绝对准确率方面, MLM模型几乎立即开始优于LTR模型。

C.2 不同掩码操作的消融研究

在3.1节中, 我们提到BERT使用混合策略来掩盖目标token, 当使用掩码语言模型 (MLM) 目标的预训练。以下是一个消融研究, 用于评估不同掩码策略的影响。

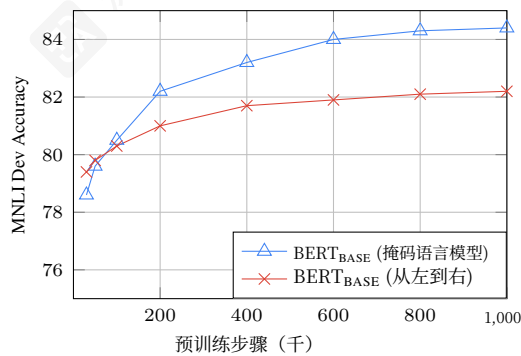


图 5: 训练步数的消融。这显示了在微调后 MNLI 准确率, 从预训练了 k 步的模型参数开始。x 轴是 k 的值。

请注意, 掩码策略的目的是减少预训练和微调之间的不匹配, 因为 [MASK] 符号在微调阶段永远不会出现。我们报告了MNLI和NER的Dev结果。对于NER, 我们报告了微调 and 基于特征的方法, 因为我们预计对于基于特征的方法, 不匹配将会被放大, 因为模型将没有机会调整表示。

掩码率			开发集结果		
MASK	SAME	RND	MNLI 微调 基于特征	NER	
80%	10%	10%	84.2	95.4	94.9
100%	0%	0%	84.3	94.9	94.0
80%	0%	20%	84.1	95.2	94.6
80%	20%	0%	84.4	95.2	94.7
0%	20%	80%	83.7	94.8	94.6
0%	0%	100%	83.6	94.9	94.6

表8: 不同掩码策略的消融实验。

结果展示在表8中。在表格中, MASK表示我们将目标词替换为 [MASK] 符号用于 MLM; SAME表示我们保留目标词; RND表示我们将目标词替换为另一个随机词。

表格左侧的数字表示MLM预训练期间使用的特定策略的概率 (BERT使用80%, 10%, 10%)。右侧部分表示Dev集结果。对于基于特征的方法, 我们将BERT的最后4层作为特征进行拼接, 这在第5.3节中显示为最佳方法。

从表格中可以看出, 微调对不同掩码策略具有令人惊讶的鲁棒性。然而, 正如预期的那样, 当将基于特征的方法应用于NER时, 仅使用MASK策略存在问题。有趣的是, 仅使用RND策略的表现也比我们的策略差得多。